UNO Rulebook RAG Chatbot

Technical report: architecture, data flow, APIs, front end, security, deployment, setup and evaluation

Item	Details
Live site	https://uno-chatbot.buildwithmayank.tech (chat page: /uno-rules-chat/)
Backend	Google Cloud Run service `uno-rag-backend` (us-central1)
Stack	Python 3.13 · FastAPI · LangGraph · ChromaDB + BM25 · pdfplumber · Gemini on Vertex AI · WordPress plugin · Caddy · Cloudflare · Langfuse
Version	2.0 (supersedes the first, Ollama-based report of 29 October 2026, kept in docs/archive/)
Prepared by	Mayank Bisht
Date	1 November 2026
Evaluation	Local run 2026-11-01 03:00, production run 2026-11-01 03:06, security checks 2026-11-01 02:54

Contents

1. Executive summary 4
2. System architecture 5
2.1 How data flows 6
3. Project structure 8
3.1 Role of each file 8
3.2 Generated and local-only folders 10
4. Data ingestion and databases 11
4.1 Steps 11
4.2 Corpus 12
4.3 Vector database schema 12
4.4 Data stores overview 13
4.5 Vector database (ChromaDB) schema 14
4.6 Backend caches and usage counter 16
4.7 Conversation memory (LangGraph state) 17
4.8 WordPress database (MariaDB) 17
4.9 Other stores 18
5. Retrieval 19
6. Conversation flow (LangGraph) 21
6.1 State, memory and safeguards 22
7. What happens when someone starts a chat 23
7.1 Step by step 23
7.2 Feedback 27
7.3 Typical timings (deployed) 27
8. Front end (WordPress) 28
8.1 How the site was built 28
8.2 Plugin internals 29
8.3 Chat widget behaviour (chatbot.js) 29
8.4 Plugins, modules and libraries used 29
9. Front-end ↔ back-end communication and APIs 31
9.1 WordPress REST API (namespace uno-chat/v1) 31
9.2 Backend API (FastAPI on Cloud Run) 31
9.3 Status codes across the layers 32
10. Security 33
10.1 How API keys and secrets are protected 34
10.2 Network and DDoS protection 35
10.3 Rate limiting 35
10.4 Real visitor IPs and spoofing 36
10.5 Application hardening 36

10.6 Identity and least privilege 36
10.7 Privacy 37
10.8 Threats and mitigations 37
10.9 Security test results 37
10.10 Known limitations and recommendations 38
11. Deployment 39
11.1 Backend on Cloud Run 40
11.2 WordPress on Compute Engine 42
11.3 Cloudflare configuration 43
11.4 Updates and rollback 43
11.5 Cost 43
12. Monitoring and observability 44
13. Evaluation 45
13.1 Retrieval 45
13.2 End-to-end accuracy 45
13.3 Latency 46
13.4 Project evolution 46
14. Setting up the project from scratch 47
14.1 Prerequisites 47
14.2 Local backend 49
14.3 Local WordPress 49
14.4 Backend on Cloud Run 49
14.5 WordPress on Compute Engine with Cloudflare 49
14.6 Verify 49
15. Operations and troubleshooting 50
Appendix A. Configuration reference (.env) 51
Appendix B. Per-question results 52
Appendix C. Diagram sources 54

1. Executive summary

The UNO Rulebook RAG Chatbot answers natural-language questions about five UNO rulebooks: Classic UNO, UNO Flip, UNO Dare, UNO No Mercy and UNO All Wild. It answers only from the rulebook PDFs and shows the rulebook and page behind each answer. Questions outside the rulebooks are politely refused, and small talk is answered without searching.

Visitors use a WordPress site. A custom WordPress plugin provides the chat widget and forwards each question server-side to a Python backend (FastAPI + LangGraph) on Google Cloud Run. The backend runs a retrieval-augmented generation (RAG) flow:

1. **Route:** a small Gemini model classifies the message and rewrites follow-ups into a standalone question.
2. **Retrieve:** a hybrid keyword and vector search picks the 6 most relevant rulebook passages.
3. **Answer:** a larger Gemini model answers from those passages only.

Every turn is traced in Langfuse, and visitors can rate answers with thumbs up/down.

Headline results, all measured on the actual system (section 13):

Measure	Result	Notes
Rulebook answers correct (local / deployed)	31/31 / 31/31	Answer contains every required fact
Correct rulebook cited	100% / 100%	Source chips name the right PDF
Off-topic refused, small talk, follow-ups	6/6, 16/16, 4/4	Same results on the deployed API
Correct rulebook ranked first by retrieval	96.8%	Answer passage in the top 6: 96.8%
Median answer time, rulebook question	2.4 s local, 2.5 s deployed	First version (local Ollama model): 8.9 s
Automated security checks	26 passed, 0 failed	4 informational findings (section 10.9)
Running cost	about $18/month + Gemini usage	VM, static IP and disk; Cloud Run stays in its free tier

The system has six layers of protection:

- **Cloudflare** in front of the site.
- **A firewall** that admits only Cloudflare.
- **Per-IP rate limits** in Caddy.
- **WordPress nonce checks and question limits.**
- **A secret API key** between WordPress and the backend.
- **A least-privilege Google service account** for the Gemini calls.

API keys never reach the browser.

2. System architecture

The system has four runtime parts. The **front end** is WordPress on a Compute Engine VM, behind Cloudflare and Caddy. The **backend** is FastAPI + LangGraph on Cloud Run. The **models** are Gemini on Vertex AI. **Monitoring** is Langfuse Cloud. Figure 1 shows how they connect.

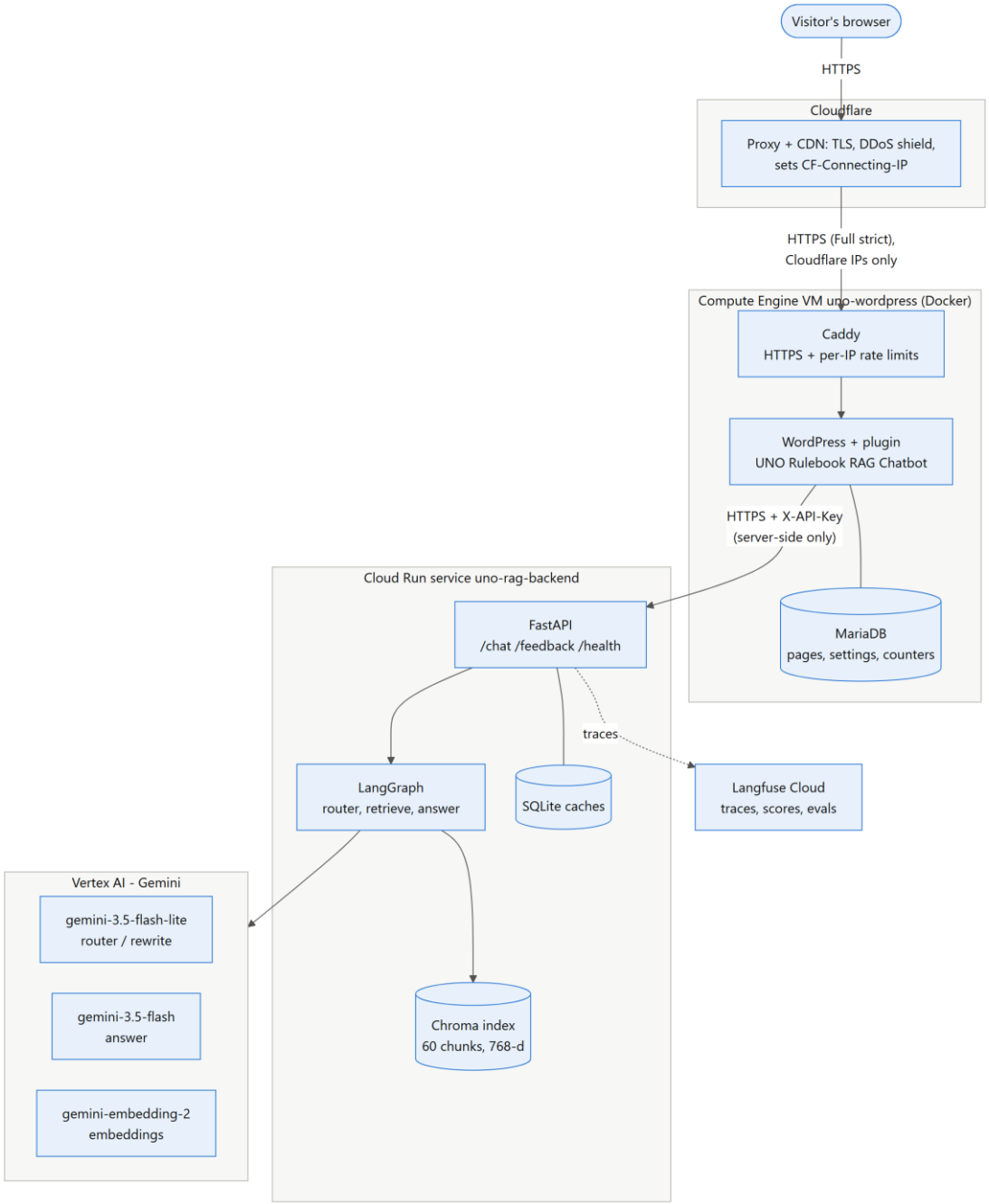


Figure 1. System architecture

Component	Technology	Runs on	Responsibility
Edge	Cloudflare (free plan)	Cloudflare network	DNS, TLS to visitors, DDoS protection, caching of static files, real visitor IP header
Web server	Caddy 2 + caddy-ratelimit	VM (Docker)	HTTPS certificate (Let's Encrypt), per-IP rate limits, reverse proxy to WordPress

Component	Technology	Runs on	Responsibility
Website	WordPress 6 (PHP 8.3), theme Twenty Twenty-Five	VM (Docker)	Home page, chat page, admin; hosts the plugin
Chat plugin	Custom plugin uno-rag-chatbot 1.6.0	Inside WordPress	Chat widget (JS), REST proxy to the backend, nonce check, question limits, settings page
Database	MariaDB 11	VM (Docker)	WordPress content, plugin settings, rate-limit counters
API	FastAPI + Uvicorn	Cloud Run	/chat, /feedback, /health; API-key check, validation, answer cache, tracing
RAG flow	LangGraph	Cloud Run	Routing, retrieval, grounded answering, query rewrite, conversation memory
Search index	ChromaDB (cosine HNSW) + rank_bm25	Cloud Run (baked into the image)	60 rulebook chunks with 768-d embeddings; keyword index built in memory
Models	gemini-3.5-flash-lite, gemini-3.5-flash, gemini-embedding-2	Vertex AI (global)	Router/rewrite, answers, embeddings
Monitoring	Langfuse Cloud (Hobby)	SaaS (EU)	Traces, user-feedback scores, evaluation datasets and experiments

2.1 How data flows

Data moves in two phases (Figure 2):

- **Indexing** runs offline when the rulebooks change. It turns the PDFs into embedded chunks in Chroma.
- **Answering** runs for every question. It reads the index and produces an answer with its sources.

Each answer also feeds the answer cache, so a repeated first question costs no model calls. It also produces a Langfuse trace, which later receives the visitor's thumbs up/down.

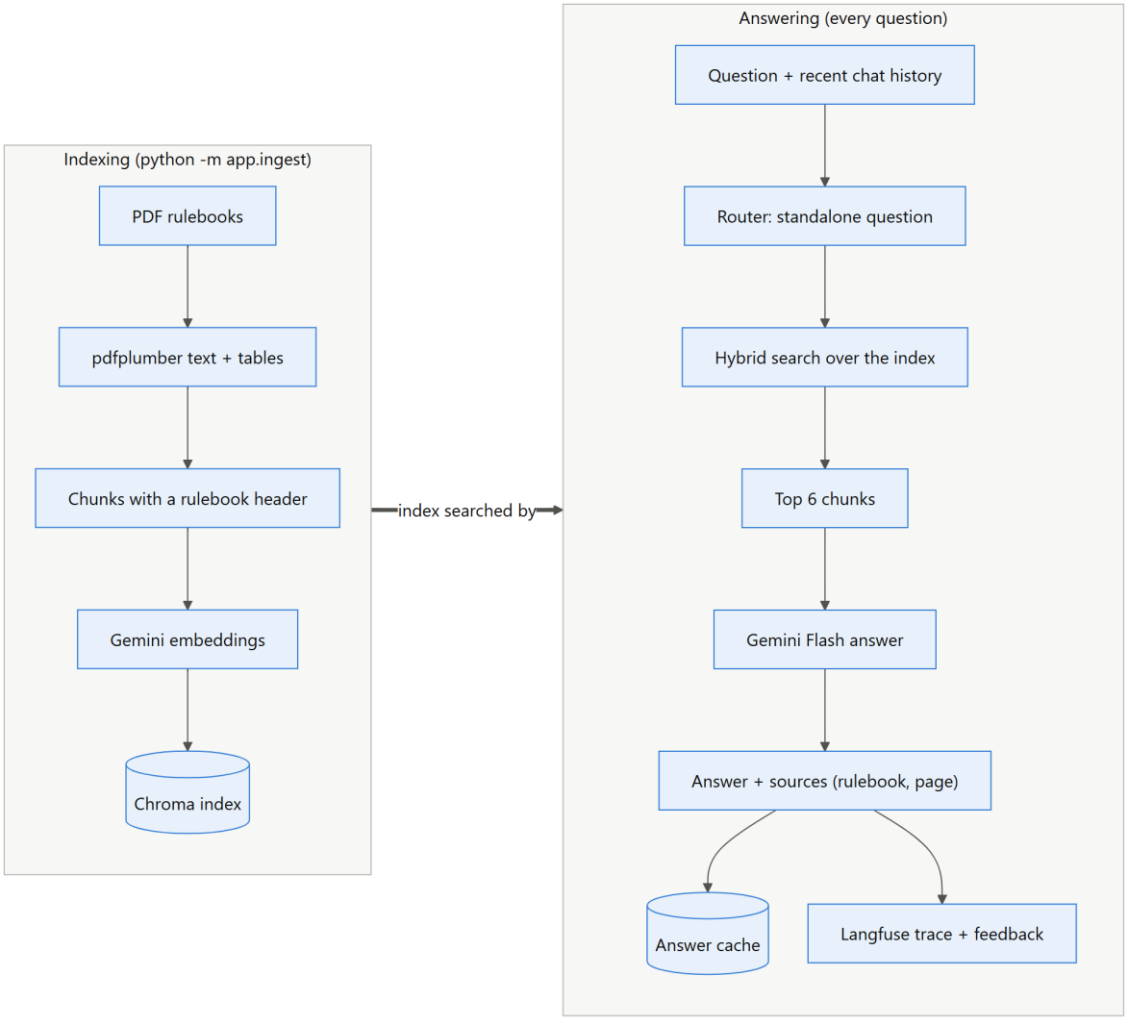


Figure 2. Data flow: indexing and answering

Where data is stored (schemas in sections 4.4–4.9):

Data	Stored in	Lifetime
Rulebook PDFs	Data/ in the repository and in the backend image	Permanent
Chunks + embeddings	chroma_db/ locally; copied into the Cloud Run image	Until the next ingest
Conversation memory	LangGraph MemorySaver (RAM of the Cloud Run instance), keyed by session_id	Until the instance stops (about 15 minutes idle) or New chat
Answer cache, query-embedding cache, usage counter	SQLite/JSON in CACHE_DIR (/tmp on Cloud Run)	Per instance on Cloud Run; persistent locally
Session id	Browser localStorage key uno_chat_session	Until New chat or cleared storage
Plugin settings, rate-limit counters, daily question count	WordPress options/transients in MariaDB	Settings permanent; counters 1 hour / 1 day
Traces and feedback	Langfuse Cloud	Per Langfuse plan retention

3. Project structure

The repository holds the backend, the WordPress plugin, both deployments, the evaluation suite and this documentation. Cloud Run builds the backend image from the repository root, so the files that control that build sit at the root.

```
RAG Practice project/
├── app/                        FastAPI + LangGraph backend (Python package)
├── Data/                       the five rulebook PDFs
├── eval/                       evaluation: dataset, runners, results
├── wordpress/
│   ├── uno-rag-chatbot/        WordPress plugin (PHP + JS + CSS)
│   ├── setup/pages.php         creates Home page, chat page, menu
│   ├── setup.ps1               local WordPress in Docker
│   └── build_zip.py            packages the plugin
├── deploy/
│   ├── cloudrun/               backend deployment scripts
│   └── wordpress/              VM deployment (Docker, Caddy, scripts)
├── docs/                       this report, diagrams, report tooling, archive
├── Dockerfile                  backend image
├── .dockerignore / .gcloudignore   what goes into the image / upload
├── docker-compose.yml          local WordPress stack
├── requirements.txt            Python dependencies
├── .env.example                configuration template (.env is git-ignored)
└── README.md                   quick-start guide
```

3.1 Role of each file

Backend: app/

File	Role
`app/main.py`	FastAPI app. Routes `/`, `/health`, `/chat`, `/feedback`, `DELETE /chat/{id}`, `/ingest`. Request/response schemas, `X-API-Key` check, answer cache, one Langfuse trace per turn, startup (graph, cache, auto-ingest) and per-request trace flush on Cloud Run.
`app/graph.py`	LangGraph state machine: `analyze` → `respond_direct` or `retrieve` → `answer` → optional `rewrite_query`. Holds the state schema, routing functions, follow-up guard and the MemorySaver checkpointer.
`app/prompts.py`	Prompt templates: ANALYZE (route + standalone question + small-talk reply, JSON), ANSWER (grounded JSON answer, quote numbers verbatim), REWRITE; fixed refusal texts.
`app/llm.py`	Gemini integration: Vertex AI client, chat models per role (router/answer), retries on 429/5xx, auth error help, daily request budget (UsageTracker), SQLite key-value caches, gemini-embedding-2 embeddings with prefixes and parallel requests, JSON parsing, health status.
`app/vectorstore.py`	Chroma vector store, in-memory BM25 index, rulebook alias detection (`detect_sources`), hybrid search with reciprocal rank fusion.
`app/ingest.py`	Indexing: pdfplumber extraction (text + tables), cleaning, duplicate-page removal, chunking, contextual header, ids, writing to Chroma. CLI: `python -m app.ingest --reset`.
`app/observability.py`	Langfuse client, LangChain callback handler, `trace_turn` (one root trace per chat turn with tags), embedding traces, scores, auth check, flush.
`app/formatting.py`	Removes Markdown from model output so the widget shows clean plain text.
`app/model_status.py`	CLI that lists the Gemini models available to the project and checks the configured ones (metadata only, no cost).
`app/config.py`	All settings (pydantic-settings) loaded from `.env`, with path helpers.

Evaluation: eval/

File	Role
`eval/dataset.py`	Hand-labelled test set: 31 rulebook questions (required facts, source PDF, evidence text), 6 off-topic, 16 small-talk, 4 two-turn follow-ups.
`eval/run_eval.py`	Retrieval metrics for 5 search methods and end-to-end accuracy/latency on the local graph (Langfuse experiment); `--backend-url` runs the same questions against the deployed API.
`eval/security_checks.py`	Automated checks of the live security measures (auth, firewall, Cloudflare, nonce, rate limits, secret exposure).
`eval/results_gemini.json`	Latest local results (retrieval + end-to-end).
`eval/results_production.json`	Latest results against Cloud Run.
`eval/results_security.json`	Latest security-check results.

WordPress plugin: wordpress/uno-rag-chatbot/

File	Role
`uno-rag-chatbot.php`	Plugin main file: settings page (Settings → UNO Chatbot), `[uno_chatbot]` shortcode, floating bubble, script/style loading with REST URL + nonce, REST routes `uno-chat/v1/ask|feedback|reset`, nonce check, per-IP and site-wide limits, server-side calls to the backend with `X-API-Key`.
`includes/home.php`	`[uno_chatbot_home]` landing page: hero, rulebooks covered, how it works, features, link to the chat page.
`assets/chatbot.js`	Chat widget: session id in localStorage, welcome text, suggestion buttons, sending questions, typing indicator, answer bubbles (textContent only), source chips, thumbs up/down, New chat.
`assets/chatbot.css`	Widget styles: dark ember theme, inline/page/floating layouts.
`assets/home.css`	Landing page styles.
`uninstall.php`	Deletes the plugin settings and daily counters when the plugin is deleted.
`readme.txt`	WordPress plugin readme and changelog.

WordPress setup and packaging: wordpress/

File	Role
`wordpress/setup/pages.php`	Creates/updates the chat page, the Home page (front page) and the Home-only navigation menu. Used by both the local and the cloud setup (`wp eval-file`).
`wordpress/setup.ps1`	Local setup: starts Docker WordPress, installs it, activates the plugin, saves settings, runs pages.php.
`wordpress/build_zip.py`	Packages the plugin to dist/uno-rag-chatbot.zip for upload to any WordPress site.
`docker-compose.yml (root)`	Local development stack: WordPress, MariaDB and a wp-cli tool container; the plugin folder is mounted live.

Deployment: deploy/

File	Role
`deploy/cloudrun/deploy.ps1`	Backend deployment: enables APIs, service account + Vertex AI User role (skippable), prepares index and env file, `gcloud run deploy --source .`, deletes the env file.
`deploy/cloudrun/prepare_index.py`	Copies only the active Chroma collection (with its embeddings) into deploy/build/chroma_db for the image.

File	Role
`deploy/cloudrun/make_env.py`	Turns `.env` into the Cloud Run env-vars file with Cloud Run overrides (cache in /tmp, production tracing).
`deploy/wordpress/deploy.ps1`	VM deployment: static IP, firewall, VM, Cloudflare checks, bundle upload, runs vm-setup.sh, firewall lock to Cloudflare, fetches admin credentials.
`deploy/wordpress/vm-setup.sh`	Runs on the VM: Docker, swap, DB passwords, `docker compose up --build`, WordPress install, plugin settings, pages.
`deploy/wordpress/docker-compose.yml`	Production stack: MariaDB (memory-tuned), WordPress (no public port), Caddy (80/443), wp-cli; log caps; real-IP config.
`deploy/wordpress/Caddyfile`	HTTPS site, trusted Cloudflare ranges, per-IP rate-limit zones, xmlrpc block, reverse proxy with X-Real-IP.
`deploy/wordpress/caddy/Dockerfile`	Builds Caddy with the caddy-ratelimit module.
`deploy/wordpress/plugin-options.php`	Saves the backend URL and key (read from stdin) into the plugin settings, keeping admin changes.

Root files and documentation

File	Role
`Dockerfile`	Backend image: python:3.13-slim, dependencies, app + PDFs + pre-built index, non-root user, Uvicorn on $PORT.
`.gcloudignore` / `.dockerignore`	Keep secrets and unrelated folders (.env, .venv, WordPress, docs, eval) out of the Cloud Run upload and image.
`requirements.txt`	Python dependencies (section 8.4).
`.env.example`	Template of every setting with comments; copy to `.env`.
`.gitignore`	Excludes secrets, virtualenv, generated index/caches, build output, credentials.
`README.md`	Quick start, deployment commands, API and configuration reference.
`docs/`	This report (.docx/.pdf), diagrams (Mermaid sources + PNG), tools to regenerate them, archive of the first report.

3.2 Generated and local-only folders

Folder / file	Created by	In git?
.venv/	python -m venv	No
.env	copied from .env.example	No (secrets)
chroma_db/	python -m app.ingest	No (rebuildable)
cache/	backend at runtime (answer + embedding caches, usage counter)	No
deploy/build/	prepare_index.py during a Cloud Run deploy	No
dist/	wordpress/build_zip.py	No
wordpress/admin-credentials*.txt	setup.ps1 / deploy.ps1	No (passwords)
docs/tools/node_modules/	npm install (report tooling)	No

4. Data ingestion and databases

`python -m app.ingest --reset` turns the PDFs in `Data/` into searchable chunks (Figure 3). Ingestion runs locally. The finished index is copied into the Cloud Run image, so the deployed backend never re-embeds the PDFs on start-up.

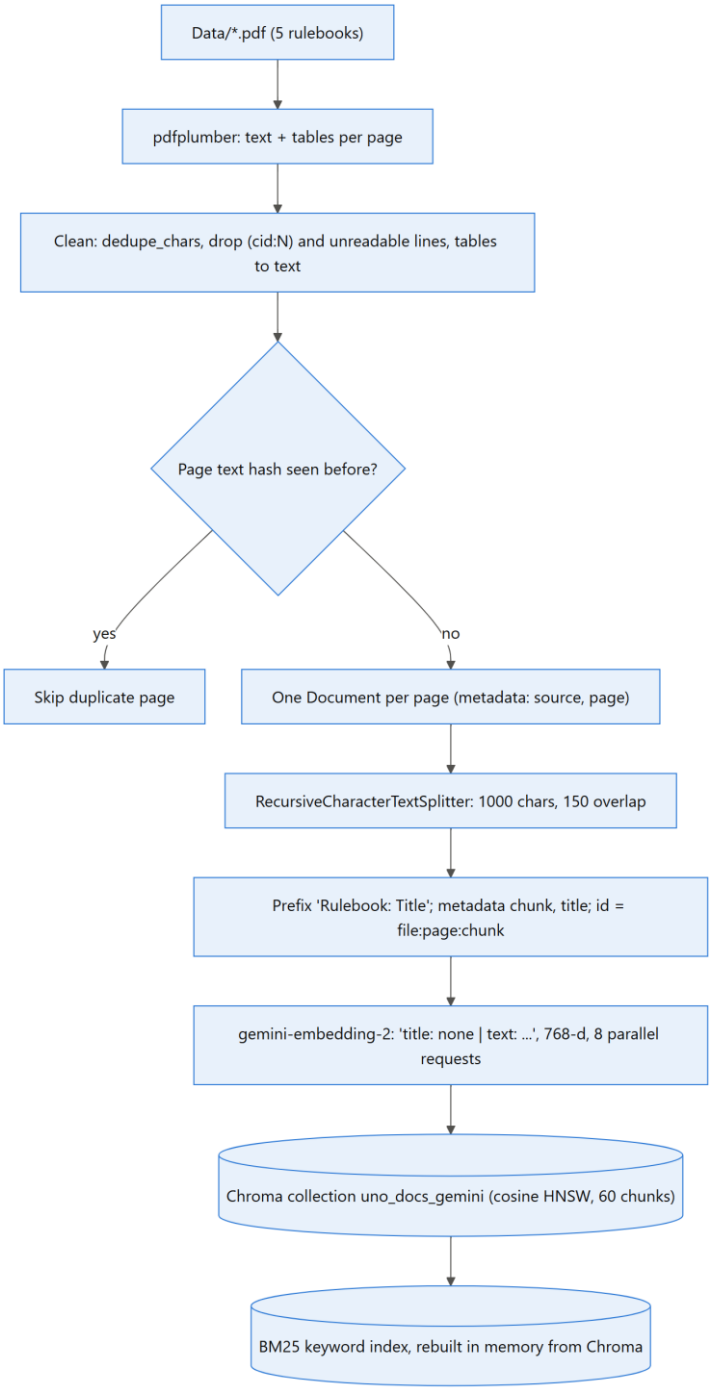


Figure 3. Ingestion pipeline

4.1 Steps

1. **Extract:** pdfplumber reads each page's text and tables. Tables become text rows joined with `|`.
2. **Clean:** extraction noise is removed (details below the list).

3. **Drop duplicates:** a page whose cleaned text hashes to one already seen is skipped (some PDFs repeat pages).
4. **Chunk:** `RecursiveCharacterTextSplitter` with 1000 characters and 150 overlap. Chunks average about 860 characters.
5. **Add context:** every chunk starts with `Rulebook: <Title>`, so both the embedding and the answer model know which edition it belongs to. Each chunk gets the id `file:page:chunk` and the metadata `source`, `page`, `chunk` and `title`.
6. **Embed:** gemini-embedding-2 at 768 dimensions. This model takes one text per request and ignores the task type, so the task is given as a text prefix (`title: none | text:` for documents) and 8 requests run in parallel.
7. **Store:** Chroma collection `uno_docs_gemini` (cosine distance, HNSW index). The BM25 keyword index is built in memory from the same chunks when the backend starts.

The cleaning in step 2 does three things:

- Collapses doubled glyphs (`dedupe_chars`).
- Removes `(cid:N)` font codes.
- Drops lines that are mostly unreadable symbols.

4.2 Corpus

Rulebook (file)	Pages kept	Chunks	Alias used for filtering
UNO Flip (uno flip.pdf)	1	29	flip
UNO No Mercy (Uno No mercy.pdf)	1	11	no mercy
UNO Dare (Uno Dare.pdf)	1	10	dare
UNO All Wild (uno-all-wild-rules-…pdf)	3	7	all wild
Classic UNO (uno classic.pdf)	1	3	classic
Total	7	60	

4.3 Vector database schema

Chroma stores everything in `chroma_db/chroma.sqlite3` plus one HNSW index folder for the collection:

Field	Type	Example	Purpose
id	string	uno flip.pdf:1:4	Stable chunk id (file:page:chunk); re-ingest overwrites
document	text	Rulebook: Uno Flip\nSKIP EVERYONE: …	Chunk text with contextual header
embedding	float[768]	[0.012, -0.044, …]	gemini-embedding-2 vector (cosine)
metadata.source	string	uno flip.pdf	Source file; shown to users as the citation
metadata.page	int	1	Page number in the PDF (shown as p.1)
metadata.chunk	int	4	Position of the chunk within its page
metadata.title	string	Uno Flip	Rulebook title; basis for the alias used by the title filter
collection metadata	map	hnsw:space = cosine	Distance function of the HNSW index

4.4 Data stores overview

The system keeps data in six places. Figure 4 shows each store, where it lives and how data moves between them.

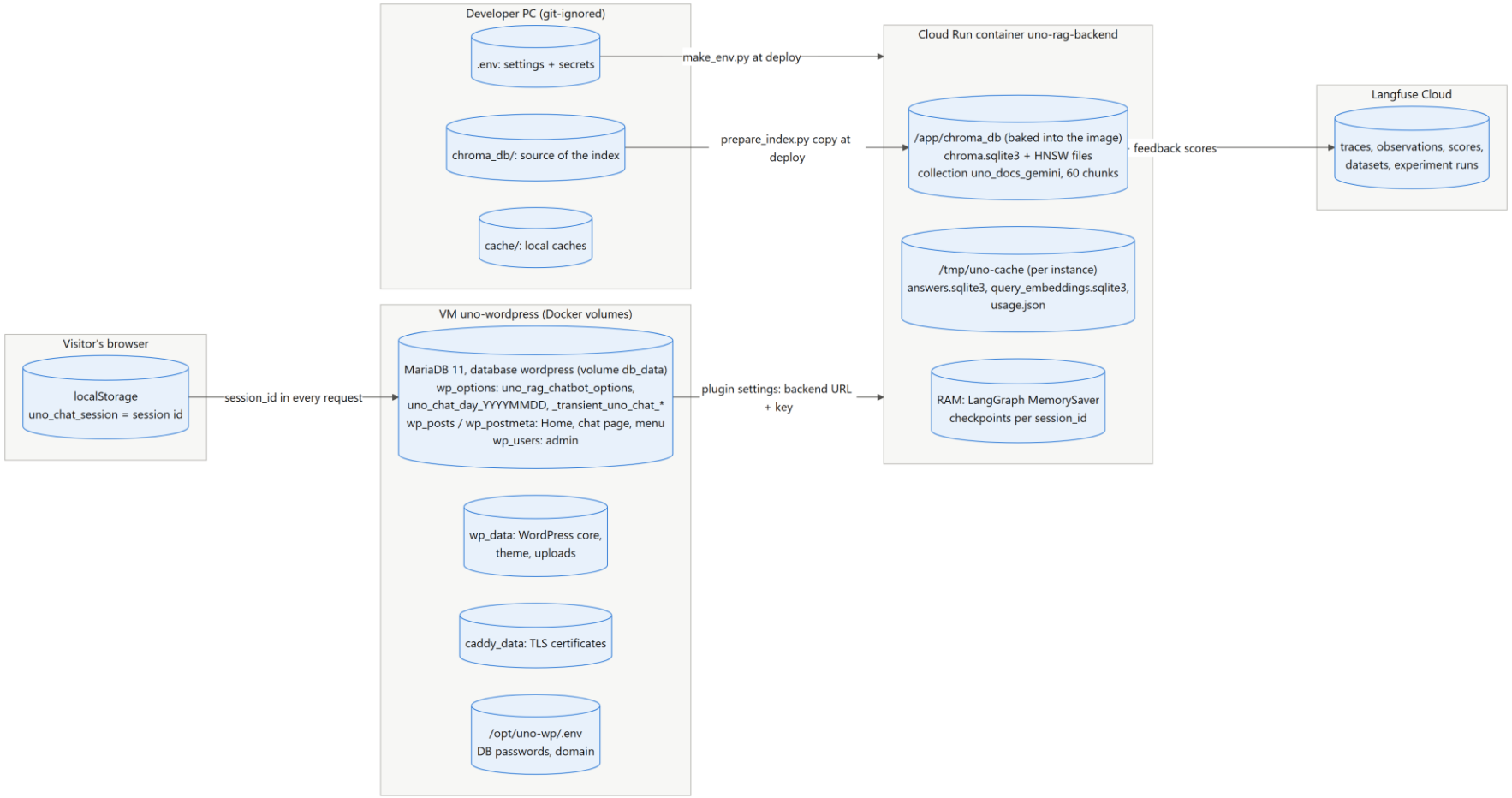


Figure 4. All data stores and how data moves between them

Store	Technology	Location	Contents	Lifetime / backup
Vector database	ChromaDB 1.x (SQLite + HNSW)	chroma_db/ on the PC; /app/chroma_db in the Cloud Run image	60 chunks, embeddings, metadata	Rebuilt by ingest; read-only copy baked into each image
Answer cache	SQLite table kv	cache/answers.sqlite3 (PC); /tmp/uno-cache (Cloud Run)	First-turn rulebook answers + sources	Per Cloud Run instance; persistent on the PC
Query-embedding cache	SQLite table kv	cache/query_embeddings.sqlite3	768-d query vectors	Same as above
Usage counter	JSON file	cache/usage.json	Model requests today	Resets each UTC day (and per instance)
Conversation memory	LangGraph MemorySaver	RAM of the backend process	Graph state per session_id	Until restart / scale to zero / New chat
Website database	MariaDB 11	Docker volume db_data on the VM	WordPress content, plugin settings, limits	Persistent; back up with VM disk snapshots
Browser storage	localStorage	Visitor's browser	Session id	Until New chat or cleared
Monitoring store	Langfuse Cloud	SaaS (EU)	Traces, scores, datasets	Langfuse plan retention

4.5 Vector database (ChromaDB) schema

Chroma runs embedded in the backend process (`chromadb.PersistentClient`); there is no separate database server. One folder holds everything:

```
chroma_db/
├── chroma.sqlite3            catalogue, chunk ids, metadata, text,
│                             full-text index, write-ahead queue
└── <segment UUID>/          HNSW vector index (uno_docs_gemini)
    ├── data_level0.bin       768-d vectors + base graph layer
    ├── link_lists.bin        upper graph layers
    ├── header.bin            index parameters (cosine, M, ef)
    └── length.bin            label to embedding map
```

Inside `chroma.sqlite3`, the tables relate as in Figure 5. Each collection has two segments: a **METADATA** segment (rows in SQLite) and a **VECTOR** segment (the HNSW files). Chroma keeps the two in sync through the `embeddings_queue` write-ahead log.

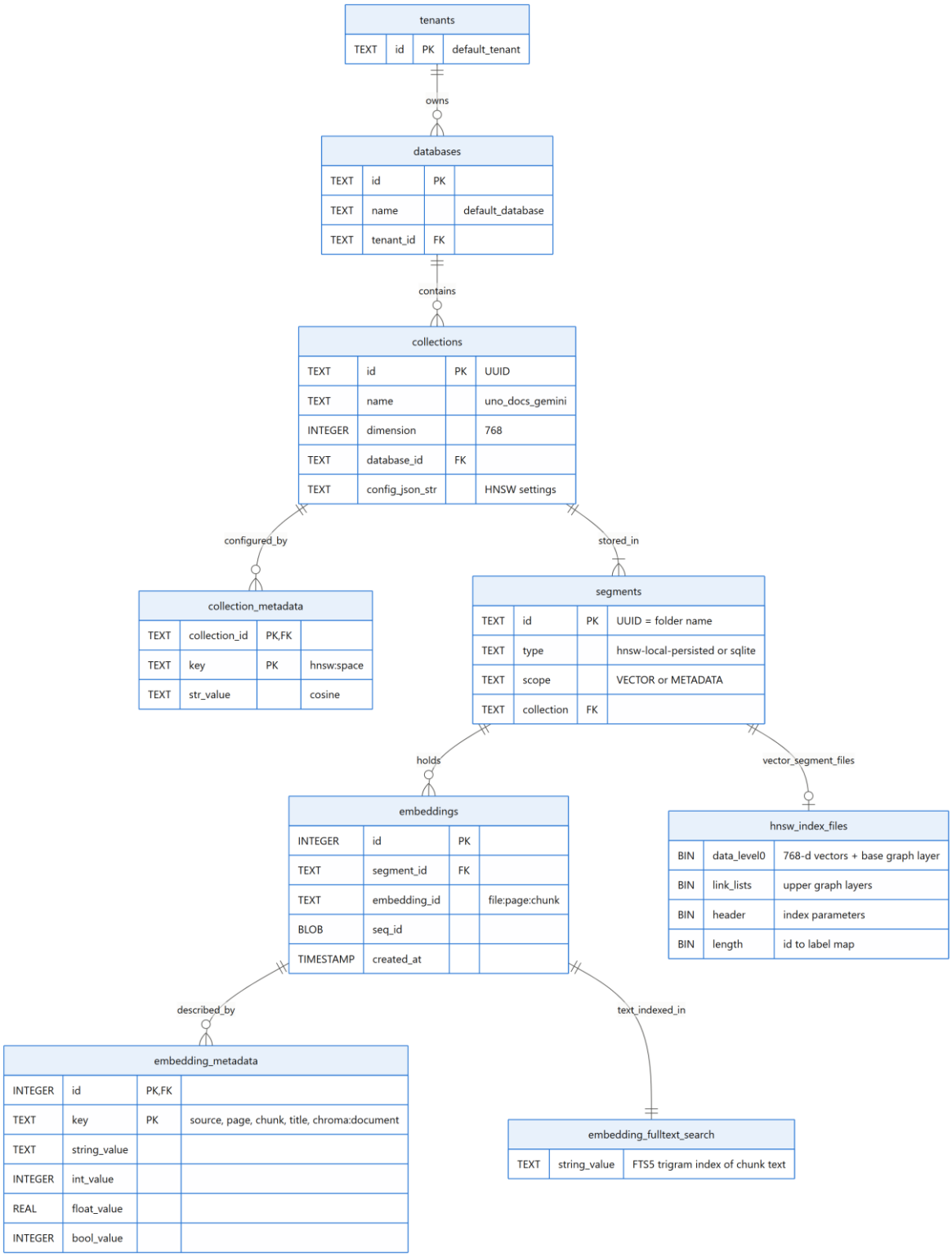


Figure 5. ChromaDB storage schema (chroma.sqlite3 + HNSW files)

Table	Rows (current)	Purpose
tenants / databases	1 / 1	Namespaces (default_tenant / default_database)
collections	1	uno_docs_gemini, dimension 768, HNSW configuration (JSON)
collection_metadata	1	hnsw:space = cosine
segments	2	One VECTOR segment (HNSW files, folder named by its UUID) and one METADATA segment (SQLite)

Table	Rows (current)	Purpose
embeddings	60	One row per chunk: internal id, segment, embedding_id (= chunk id file:page:chunk), sequence number, created_at
embedding_metadata	300	Key/value rows per chunk: source, page, chunk, title and chroma:document (the chunk text): 5 keys × 60 chunks
embedding_fulltext_search (+ _content, _data, _docsize, _idx, _config)	60	SQLite FTS5 trigram index of the chunk text (used by Chroma's where_document filters)
embeddings_queue (+ _config)	60	Write-ahead log of add/update/delete operations with the vector and metadata, applied to both segments
max_seq_id	1	Last applied queue position per segment
segment_metadata, embedding_metadata_array, maintenance_log	0	Unused here
migrations, acquire_write	18 / 11	Chroma's own schema migrations and write locks

Logical record of one chunk, as the application reads and writes it through the Chroma API:

```
id        "uno flip.pdf:1:4"             (file:page:chunk)
document  "Rulebook: Uno Flip\nSKIP ..."   (header + text, ~860 chars)
embedding [768 floats]                 (gemini-embedding-2)
metadata  {source: "uno flip.pdf", page: 1, chunk: 4, title: "Uno Flip"}
```

How the backend uses it:

- **Similarity search:** `asimilarity_search(query, k=18, filter={"source": {"$in": [...]}})`, which finds the nearest vectors (cosine) and returns them with document and metadata.
- **Keyword index:** `get(include=[documents, metadatas])` loads all 60 chunks once to build the in-memory BM25 index and the rulebook aliases.
- **Writes:** only `python -m app.ingest` (delete and re-create the collection with `--reset`, then `add_documents` with ids), or `POST /ingest`.
- **Deploy copy:** `deploy/cloudrun/prepare_index.py` copies only this collection, with its stored embeddings, into a clean folder for the image. No embedding requests are made.

4.6 Backend caches and usage counter

Both caches are small SQLite databases with one table, written by `KVCache` in `app/llm.py`:

```
CREATE TABLE kv (k TEXT PRIMARY KEY, v TEXT NOT NULL);
```

File	Key k	Value v (JSON)	Written when
answers.sqlite3	SHA-256 of router model, answer model, embed model, collection, prompt fingerprint and the normalized question (lower case, spaces collapsed, trailing ?!. removed)	{"answer": "...", "sources": [{"source": "Uno No mercy.pdf", "page": 1}]}	A first-turn rulebook question was answered from the rulebooks (found = true)
query_embeddings.sqlite3	SHA-256 of embed model, dimensions and prefix + query text	[768 floats]	A search query was embedded

Because the models, collection and prompts are part of the key, changing any of them automatically invalidates old entries. Nothing personal is stored: only questions, answers and vectors.

The daily usage counter is a small JSON file, `usage.json`:

```
{"date": "2026-11-01", "count": 512}   # requests on this UTC date
```

Every Gemini request first calls `usage.reserve()`. When `count` would pass `DAILY_REQUEST_BUDGET` (2000), the call raises `DailyLimitReached` and the API answers 429.

4.7 Conversation memory (LangGraph state)

LangGraph's `MemorySaver` keeps one checkpoint per thread (`thread_id` = the widget's `session_id`) in the backend's memory. Each checkpoint stores the graph state:

Field	Type	Meaning
messages	list[BaseMessage] (append-only)	The conversation: HumanMessage / AIMessage pairs; the last 6 are given to the router
question	str	The visitor's latest message
route	str	chitchat, rulebook or off_topic
reply	str	Small-talk reply drafted by the router
standalone_question	str	The question rewritten without chat context
search_query	str	Text sent to retrieval (changes after a rewrite)
documents	list[Document]	The 6 retrieved chunks
used_documents	list[Document]	Chunks the answer used, which become the sources
found	bool	Whether the answer model found the answer in the chunks
rewrites	int	How many query rewrites were done (max 1)

`DELETE /chat/{session_id}` (the New chat button) deletes the thread. Memory is lost when the Cloud Run instance stops. That's acceptable for short rule questions; a persistent checkpointer (for example Postgres) would keep conversations across restarts.

4.8 WordPress database (MariaDB)

WordPress uses its standard schema in the database `wordpress` with the table prefix `wp_`:

- `wp_posts`, `wp_postmeta`
- `wp_options`
- `wp_users`, `wp_usermeta`
- `wp_terms`, `wp_term_taxonomy`, `wp_term_relationships`, `wp_termmeta`
- `wp_comments`, `wp_commentmeta`
- `wp_links`

The plugin creates **no tables of its own**. It stores its data in `wp_options`, with this layout:

```
CREATE TABLE wp_options (
  option_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  option_name  VARCHAR(191) NOT NULL UNIQUE,
  option_value LONGTEXT NOT NULL,          -- PHP-serialized for arrays
  autoload     VARCHAR(20) NOT NULL DEFAULT 'yes'
);
```

option_name	option_value	Written by
uno_rag_chatbot_options	Serialized array: backend_url, api_key, title, welcome, floating (0/1), per_hour (questions per visitor per hour), per_day (site-wide daily cap)	Settings page; deploy (plugin-options.php)
uno_chat_day_YYYYMMDD	Integer: questions answered site-wide on that UTC day (autoload off; incremented in SQL with UPDATE … + 1; older days deleted)	/ask
_transient_uno_chat_rl_<md5(IP)>	Serialized array {count, reset (unix time)}: questions by one IP in the current hour	/ask
_transient_uno_chat_fb_<md5(IP)>	Same format for thumbs up/down ratings (limit 60/hour)	/feedback
_transient_timeout_uno_chat_*	Expiry time of the matching transient	WordPress

Content created by `wordpress/setup/pages.php`:

Table	Rows	Values
wp_posts	Page home	post_type page, content `[uno_chatbot_home]` shortcode block
wp_posts	Page uno-rules-chat	post_type page, content `[uno_chatbot layout="page"]` shortcode block
wp_posts	Main menu	post_type wp_navigation with a single Home link
wp_postmeta	_wp_page_template	page-no-title for both pages
wp_options	show_on_front / page_on_front	page / ID of the Home page
wp_options	permalink_structure	/%postname%/

Security notes:

- **Visitor IPs:** they appear only as MD5 hashes in transient names, and transients expire after one hour.
- **API key:** it is the one secret in the database. Only WordPress PHP and database administrators can read it.
- **Database access:** MariaDB is reachable only on the internal Docker network, with a random password generated on the VM.
- **Uninstall:** removes the plugin option and the daily counters.

4.9 Other stores

Store	Schema	Notes
Browser localStorage	uno_chat_session = random id (letters, digits, - and _)	Sent with each question; replaced by New chat
Langfuse Cloud	Trace (name chat, session, user = visitor hash, input, output, metadata, tags) → observations (graph nodes, LLM generations with model and tokens, embedding call) → scores (user-feedback BOOLEAN; eval scores); datasets uno-rulebook-qa with items and experiment runs	Written by the backend and the evaluation; nothing is read back by the app
Caddy volumes	caddy_data: certificates and ACME account; caddy_config: runtime config	Kept across redeploys so certificates are not re-requested
/opt/uno-wp/.env on the VM	DB_PASSWORD, DB_ROOT_PASSWORD, SITE_DOMAIN, CF_IP_RANGES	Mode 600, root only; generated on the VM
Repository .env	All settings in Appendix A	Git-ignored; source of the Cloud Run environment

Backup and restore:

- **Search index:** rebuildable from the PDFs with `python -m app.ingest --reset`, so it needs no backup.
- **Backend caches and counters:** disposable.
- **WordPress:** back it up with VM disk snapshots, or `wp db export` inside the wpcli container.

5. Retrieval

Retrieval combines two searches, because each catches what the other misses (Figure 6). **Vector search** finds passages that mean the same thing as the question. **BM25 keyword search** finds exact card names and numbers, such as "Wild Draw Color" or "25 cards".

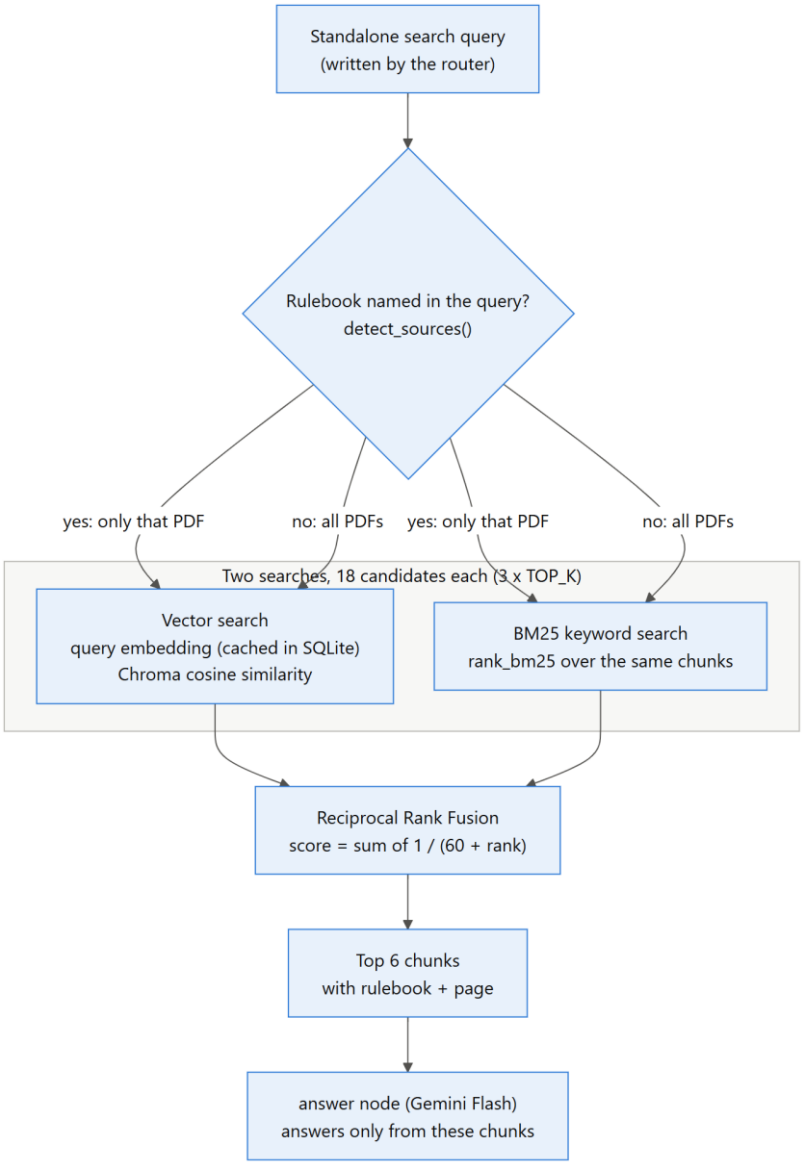


Figure 6. Hybrid retrieval with reciprocal rank fusion

1. **Rulebook filter:** if the standalone question names an edition (an alias such as "flip", "no mercy" or "classic"), both searches are restricted to that PDF.
2. **Two searches:** each returns up to 18 candidates (3 × TOP_K). The query embedding uses the prefix `task: search result | query:` and is cached in SQLite, so repeated questions cost no embedding request.
3. **Reciprocal rank fusion:** every chunk scores Σ 1 / (60 + rank) over the two result lists. Chunks found by both searches rise to the top.
4. **Top 6** chunks (TOP_K) go to the answer node with their rulebook and page.

`RETRIEVAL_MODE=bm25` turns off the vector search, which saves the embedding request.

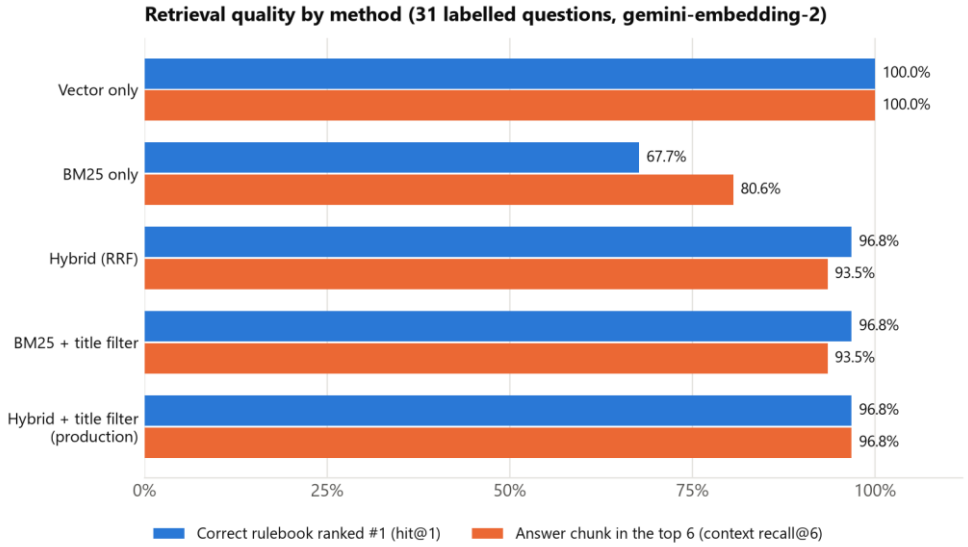


Figure 7. Retrieval quality by method (31 labelled questions)

Method	Hit@1	Recall@3	Recall@6	MRR	Avg time
Vector only	100%	100%	100%	0.952	4.4 ms
BM25 only	67.7%	61.3%	80.6%	0.526	0.3 ms
Hybrid (RRF)	96.8%	87.1%	93.5%	0.798	1.6 ms
BM25 + title filter	96.8%	77.4%	93.5%	0.663	0.2 ms
Hybrid + title filter (production)	96.8%	93.5%	96.8%	0.836	1.5 ms

The column headings mean:

- **Hit@1:** the correct rulebook is ranked first.
- **Recall@k:** the passage that contains the answer is among the first k results.
- **MRR:** the mean of 1 / rank of that passage.

> Finding: pure vector search with gemini-embedding-2 reaches 100% recall@6 and MRR 0.952, above the production hybrid (96.8%, 0.836). End-to-end accuracy is already 100%, so the hybrid stays for robustness on exact card names. A vector-first fusion weight is a candidate improvement.

6. Conversation flow (LangGraph)

Each message runs through a small LangGraph state machine (Figure 8). The graph is compiled once at start-up with a MemorySaver checkpointer. The visitor's `session_id` is the thread id, so a conversation keeps its history across turns.

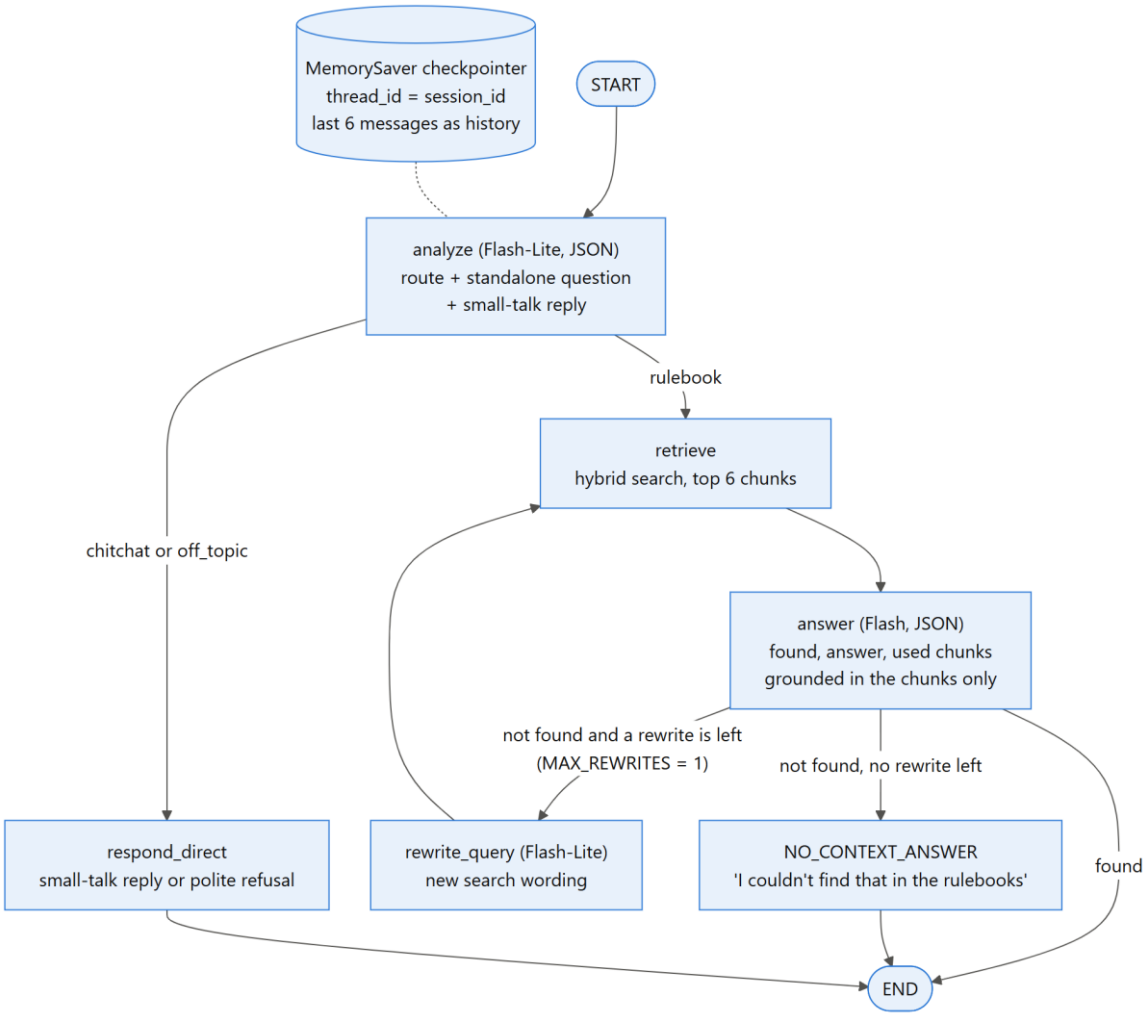


Figure 8. LangGraph conversation flow

Node	Model	What it does	Model requests
analyze	gemini-3.5-flash-lite (thinking: minimal), JSON output	Classifies the message as chitchat, rulebook or off_topic; rewrites follow-ups into a standalone question ("what about in Flip?" → full question); drafts the small-talk reply. Unknown routes default to rulebook.	1
respond_direct	–	Returns the small-talk reply, or the fixed off-topic refusal.	0
retrieve	gemini-embedding-2 (query)	Hybrid search for the standalone question (section 5).	0–1 (cached)
answer	gemini-3.5-flash (thinking: low), JSON output	Answers only from the 6 chunks, returns found/answer/used chunk numbers; the used chunks become the sources. Numbers are quoted verbatim; Markdown is stripped.	1
rewrite_query	gemini-3.5-flash-lite	If nothing relevant was found, rephrases the search once (MAX_REWRITES = 1) and retrieves again.	1 (rare)

6.1 State, memory and safeguards

The graph state holds the following fields:

- `messages`: the chat history.
- `question` and `standalone_question`.
- `route` and `reply`.
- `search_query`.
- `documents` and `used_documents`.
- `found` and `rewrites`.

Safeguards in the flow:

- **History:** the last 6 messages (HISTORY_WINDOW) are given to the router, so follow-ups are understood.
- **Follow-up guard:** a regular expression catches follow-ups ("it", "that", "what about…") that the router failed to rewrite, and keeps the previous topic in the search.
- **No guessing:** if no chunk answers the question, the reply is the fixed text "I couldn't find that in the rulebooks…" rather than an invented answer.
- **Off-topic:** questions unrelated to UNO get a fixed polite refusal, with no search and no answer model.
- **Cost:** small talk and off-topic messages cost 1 request; rulebook questions cost 3; a repeated first question costs 0 (answer cache).

7. What happens when someone starts a chat

Figures 9 and 10 (on the following landscape pages) trace one question from the visitor's browser to Gemini and back. The front-end half covers the browser, Cloudflare, Caddy and WordPress. The back-end half covers WordPress, FastAPI, LangGraph, Gemini and Langfuse.

7.1 Step by step

1. **Page load:** WordPress renders the chat page. The plugin loads `chatbot.js` and passes it the REST URL, a fresh REST nonce, the title, welcome text and three suggested questions (`wp_localize_script`). The backend URL and key are not included.
2. **Session:** the widget reads or creates a random session id in localStorage (`uno_chat_session`) and shows the welcome message and suggestion buttons.
3. **Ask:** the visitor types a question (or clicks a suggestion). The widget POSTs `{question, session_id}` to `/wp-json/uno-chat/v1/ask` with the `X-WP-Nonce` header and shows a typing indicator.
4. **Cloudflare:** terminates TLS, applies its DDoS protection and forwards the request to the VM over HTTPS, adding `CF-Connecting-IP` with the visitor's real IP.
5. **Caddy:** takes the visitor IP from that header (only because the request came from a Cloudflare IP range), applies the per-IP limits (300 requests/minute overall, 20/minute for the chat API) and proxies to WordPress with `X-Real-IP`.
6. **WordPress plugin:** checks the nonce and the question limits, then calls the backend server-side (checks listed below).
7. **Backend:** checks the API key, the request body and the daily budget, or answers from the cache (checks listed below).
8. **Graph:** analyze (Flash-Lite) → retrieve (hybrid search; the query embedding usually comes from the cache) → answer (Flash). Each step and model call is recorded in the turn's Langfuse trace, which is sent before the response ends.
9. **Response:** FastAPI returns `{answer, sources[], cached, trace_id}`. The plugin passes it to the browser unchanged except for validation of the trace id.
10. **Display:** the widget shows the answer as plain text (`textContent`, never HTML), source chips such as "Uno No mercy.pdf · p.1" and thumbs up/down buttons.
11. **Follow-up:** the next question carries the same session id, so the router sees the history and rewrites the follow-up into a standalone question.
12. **New chat:** creates a new session id and calls `/reset`, which makes the backend forget the old conversation (`DELETE /chat/{id}`).

Checks made by the WordPress plugin (step 6):

- The nonce must be valid (otherwise 403).
- At most 10 questions per hour per visitor IP.
- At most 300 questions per day for the whole site.
- The question must be 1–1000 characters, and the session id must contain only `A–Z a–z 0–9 _ -`.

The plugin then calls `POST {backend}/chat` with `X-API-Key` and an anonymous visitor id: the first 16 characters of a salted hash of the IP.

Checks made by the backend (step 7):

- The API key must match (otherwise 401).
- Pydantic validates the body (otherwise 422).

- The daily request budget must not be exhausted (otherwise 429).

If this is the first turn of the session and the question was answered before, the cached answer is returned with no model calls.

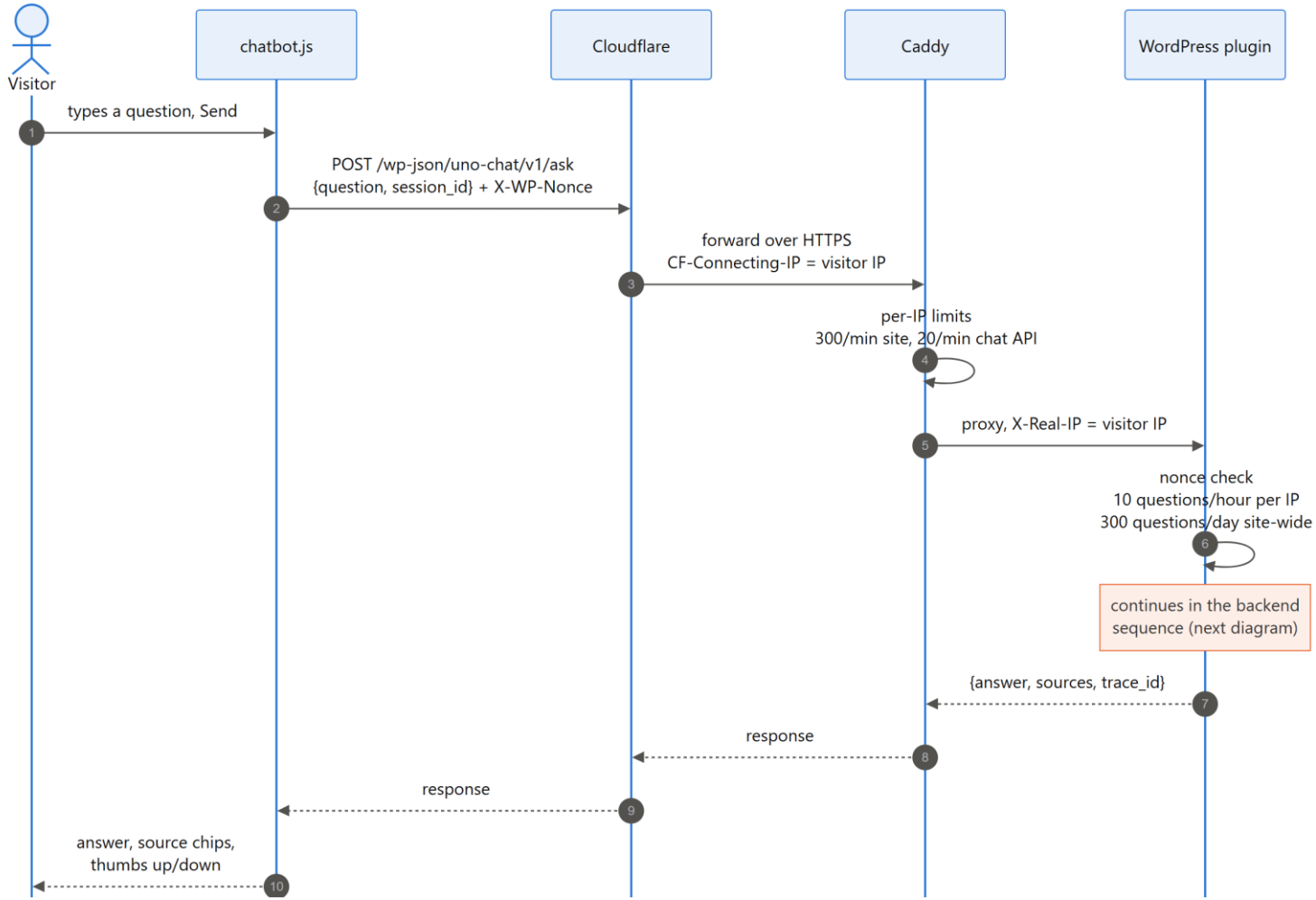


Figure 9. Chat request: browser to WordPress

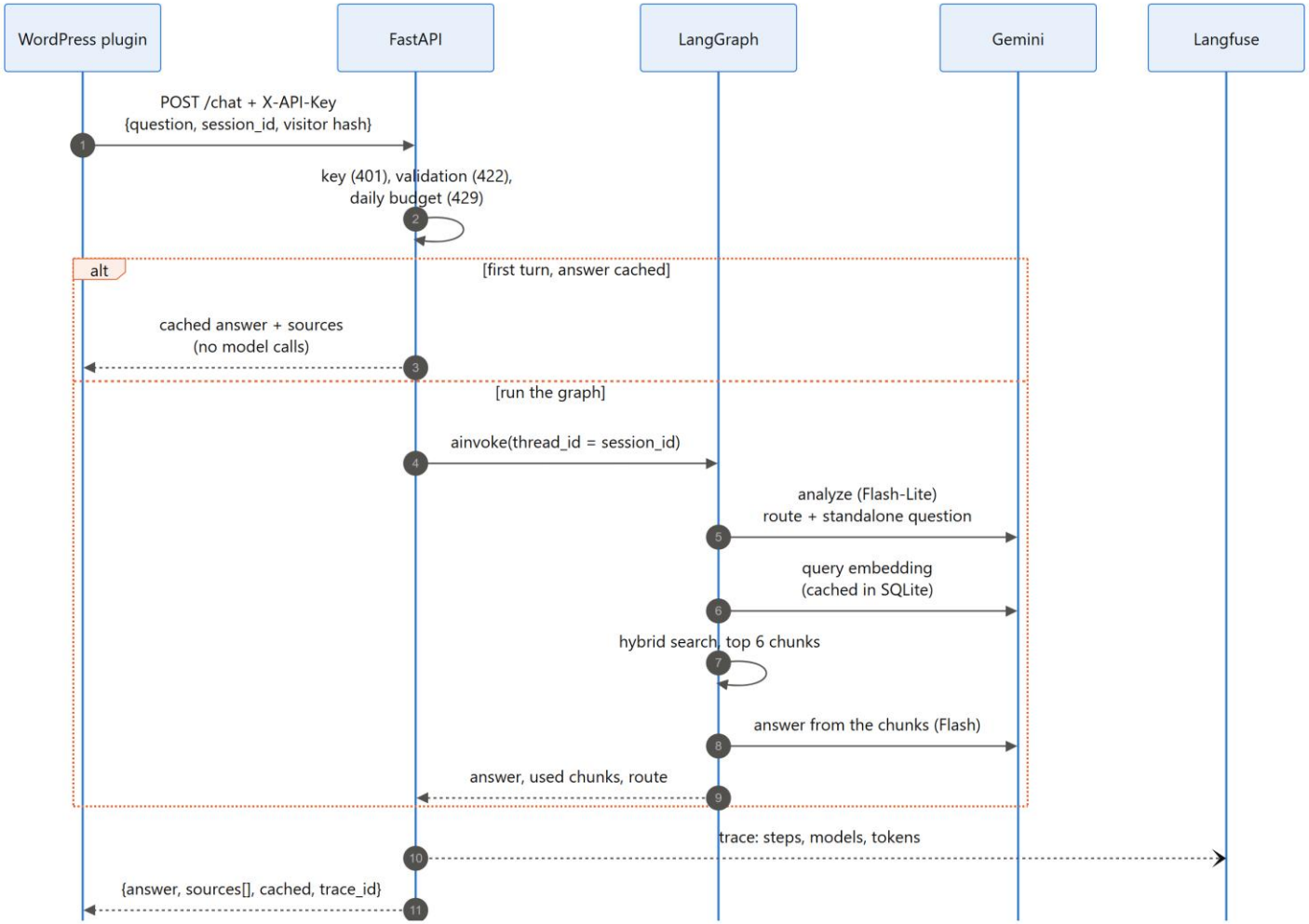


Figure 10. Chat request: WordPress to the backend and Gemini

7.2 Feedback

Clicking thumbs up or down sends the answer's `trace_id` through the same proxy. The backend stores it in Langfuse as the BOOLEAN score `user-feedback` on that answer's trace (Figure 11).

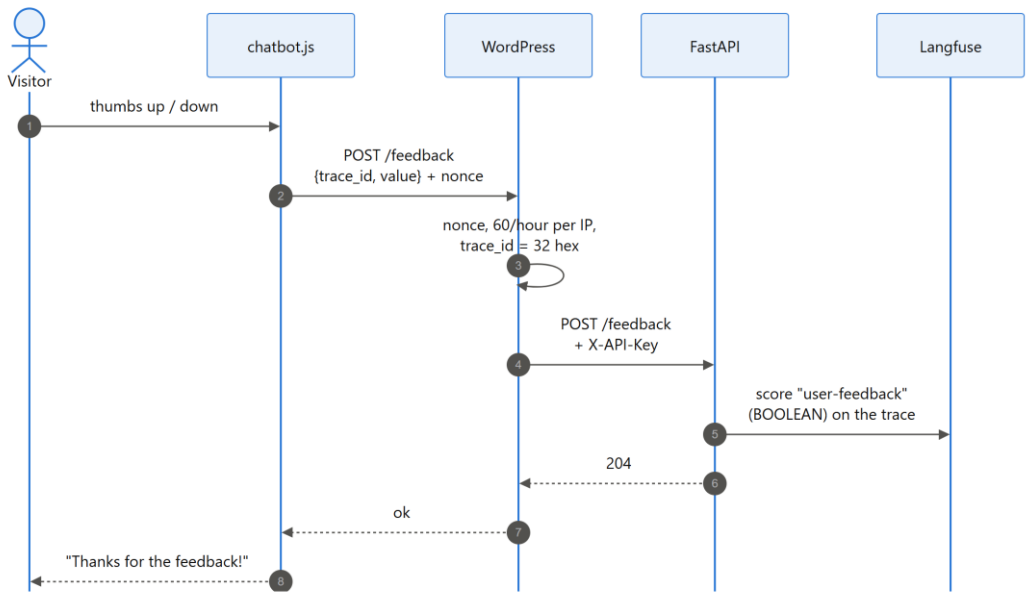


Figure 11. Feedback flow

7.3 Typical timings (deployed)

Message type	Median	95th percentile	Model requests
Small talk	1.1 s	9.7 s	1
Off-topic	1.0 s	10.3 s	1
Rulebook question	2.5 s	7.5 s	3
Follow-up (second turn)	2.5 s	3.0 s	3

These are measured at the Cloud Run API. Through WordPress and Cloudflare, add about 0.3–1 s. After about 15 idle minutes, Cloud Run starts a new instance, and the first request takes a few seconds longer.

8. Front end (WordPress)

The public site is a standard WordPress installation. It uses the default block theme **Twenty Twenty-Five**, and all chatbot functionality comes from one custom plugin, **UNO Rulebook RAG Chatbot**. There are no third-party WordPress plugins. Figure 12 shows how the pieces fit together.

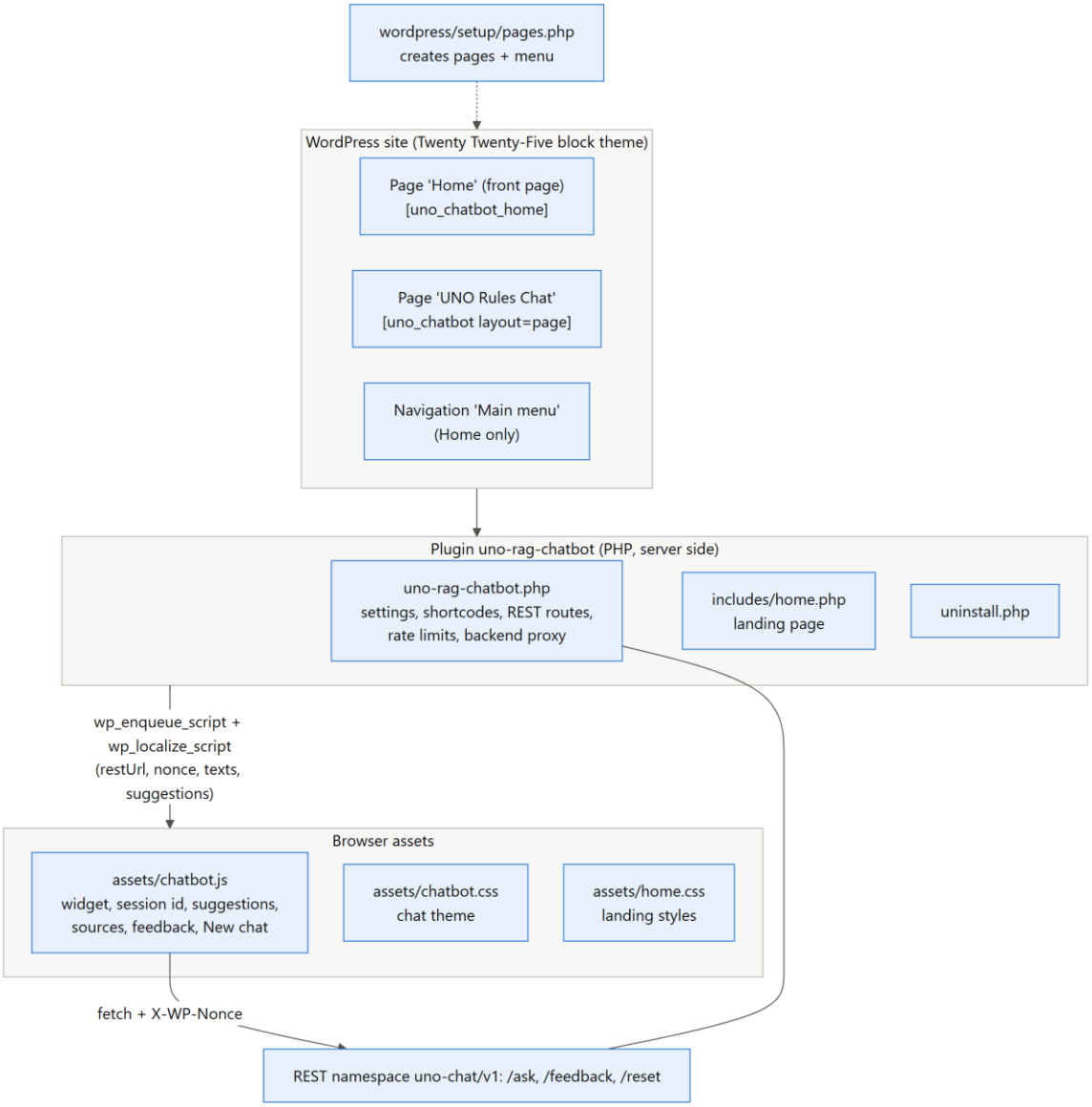


Figure 12. Front-end components

8.1 How the site was built

1. **Plugin:** the plugin (PHP, JavaScript and CSS, no build step) was written in `wordpress/uno-rag-chatbot/`, and its widget and landing page were designed to match.
2. **Install:** WordPress was installed and configured by script (`wp-cli`), locally in Docker and on the VM, so both sites are reproducible.
3. **Pages:** `wordpress/setup/pages.php` creates the two pages below and the navigation menu.
4. **Plugin settings:** the backend URL and API key are saved server-side by the deploy script.

The pages and menu:

- **Home** (`[uno_chatbot_home]`), set as the front page. It's a landing page that explains what the assistant does, which rulebooks it knows and how it works, with a "Start chatting" button.
- **UNO Rules Chat** (`/uno-rules-chat/`, `[uno_chatbot layout="page"]`). It uses the full-width "page-no-title" template with the chat box.
- **Main menu**, which contains only Home, so the theme doesn't list every page.

8.2 Plugin internals

Part	Implementation
Settings page	Settings → UNO Chatbot: backend URL, API key (password field), title, welcome text, floating bubble, questions per visitor per hour, questions per day (site). Shows backend health and today's usage.
Shortcodes	`[uno_chatbot]` (inline box; `layout="page"` adds the page header), `[uno_chatbot_home]` (landing page).
Floating bubble	Optional chat bubble on every page except pages that already show the inline chat.
Script data	`wp_localize_script` passes restUrl, nonce, texts and suggestions; never the backend URL or key.
REST routes	`uno-chat/v1/ask`, `/feedback`, `/reset`, all POST, all protected by the WordPress REST nonce, with argument validation.
Limits	Per-IP transients (10 questions/hour, 60 ratings/hour) and a site-wide daily counter in its own option, incremented atomically in SQL.
Backend calls	`wp_remote_request` with `X-API-Key`, 90 s timeout; backend errors are mapped to friendly messages (502/429).
Privacy	The backend and Langfuse only get a salted hash of the visitor IP.
Uninstall	Removes the settings and the daily counters.

8.3 Chat widget behaviour (chatbot.js)

- **Mounting:** mounts on every `[data-uno-chat]` element. The inline, page and floating layouts share one code path.
- **Header:** an avatar, the title and the subtitle "Answers from 5 UNO rulebooks", plus a **New chat** button.
- **Before the first question:** suggestion buttons; they disappear after the first question.
- **While waiting:** a typing indicator. Errors (rate limit, expired session, backend down) are shown as a red message bubble.
- **Rendering:** answers are inserted with `textContent`, so model output can never inject HTML or scripts.
- **Feedback:** source chips (rulebook · page) and thumbs up/down. Feedback is sent once per answer and confirmed with "Thanks for the feedback!".

8.4 Plugins, modules and libraries used

Component	Version / source	Used for
UNO Rulebook RAG Chatbot (custom WordPress plugin)	1.6.0, this repository	All chatbot features on the site
Twenty Twenty-Five (WordPress theme)	Bundled with WordPress	Site layout, header, footer
WordPress / PHP / Apache	wordpress:php8.3-apache image	Website runtime
MariaDB	mariadb:11 image	WordPress database
wp-cli	wordpress:cli-php8.3 image	Scripted install and configuration

Component	Version / source	Used for
Caddy + caddy-ratelimit (github.com/mholt/caddy-ratelimit)	caddy:2, built with xcaddy	HTTPS, reverse proxy, per-IP rate limits
Cloudflare (free plan)	SaaS	DNS, proxy, DDoS protection, TLS
fastapi, uvicorn[standard]	requirements.txt	HTTP API and ASGI server
langgraph, langchain-core	requirements.txt	Conversation graph, messages, prompts, checkpointer
langchain-google-genai, google-genai	≥ 4.4 / ≥ 2.20	Gemini chat models and embeddings on Vertex AI
chromadb, langchain-chroma	requirements.txt	Vector store
rank_bm25	requirements.txt	Keyword search
pdfplumber, langchain-text-splitters	requirements.txt	PDF extraction, chunking
pydantic-settings	requirements.txt	Configuration from .env
httpx	requirements.txt	HTTP client (evaluation, security checks)
langfuse	≥ 5	Tracing, scores, datasets
docx, mermaid (npm); playwright, matplotlib (Python)	docs/tools only	Building this report and its diagrams

9. Front-end ↔ back-end communication and APIs

The browser never talks to the backend. All communication follows one path: **browser → Cloudflare → Caddy → WordPress REST API → (server-to-server) → FastAPI**. As a result:

- the backend URL and its key stay on the WordPress server
- the site can apply its own limits before any model is called
- the backend needs no CORS configuration

Header	Set by	Purpose
X-WP-Nonce	chatbot.js (value from the page)	Proves the request comes from a page WordPress served recently (CSRF protection)
CF-Connecting-IP	Cloudflare	Real visitor IP; Caddy trusts it only from Cloudflare's ranges, and Cloudflare rejects client-supplied values
X-Real-IP	Caddy	Visitor IP passed to WordPress (overwrites any client value); used for per-IP limits
X-API-Key	WordPress plugin (server-side)	Shared secret required by every backend data endpoint

9.1 WordPress REST API (namespace uno-chat/v1)

Endpoint	Request body	Validation and limits	Response
POST /ask	{question, session_id}	Nonce; question 1–1000 chars; session_id [A-Za-z0-9_-]{1,100}; 10/hour per IP; 300/day site-wide	{answer, sources[{source,page}], cached, trace_id}
POST /feedback	{trace_id, value}	Nonce; trace_id 32 hex chars; value 1 or -1; 60/hour per IP	{ok: true}
POST /reset	{session_id}	Nonce; session_id format	{ok: true} (backend forgets the conversation)

9.2 Backend API (FastAPI on Cloud Run)

Endpoint	Auth	Request	Response
GET /	–	–	{service, health, docs}
GET /health	–	–	status, Gemini status and models, requests_today, daily_request_budget, documents_in_collection, Langfuse status (no secrets)
POST /chat	X-API-Key	{question: 1–1000 chars, session_id ≤ 100, visitor? ≤ 64}	{answer, sources[{source, page}], cached, trace_id}
POST /feedback	X-API-Key	{trace_id: 32 hex, value: 1 | -1, comment? ≤ 500}	204 No Content (Langfuse score user-feedback)
DELETE /chat/{session_id}	X-API-Key	–	{cleared: session_id}
POST /ingest	X-API-Key	{reset: bool}	{files, pages, chunks, collection_count}
GET /docs, /openapi.json	–	–	Interactive API documentation (schema only)

Example

```
POST /chat   X-API-Key: <secret>   Content-Type: application/json
{"question": "What is the Mercy rule in UNO No Mercy?", "session_id": "7f3c…", "visitor":
"a91be2…"}
```

```
200 OK
{"answer": "According to the Uno No Mercy rulebook, if a player ever has 25 or more cards in
their hand, they are out of the game …",
 "sources": [{"source": "Uno No mercy.pdf", "page": 1}], "cached": false, "trace_id":
"5d0c…e41a"}
```

9.3 Status codes across the layers

Code	Returned by	Meaning	What the visitor sees
403	Cloudflare	Forged CF-Connecting-IP (error 1000) or a Cloudflare security rule	Cloudflare error page
429	Caddy	Per-IP limit (300/min site, 20/min chat API, 5/min login), with Retry-After	Error message in the widget
403	WordPress	Missing/expired nonce	"Your session expired. Please reload the page."
429	WordPress	10 questions/hour per IP, or 300/day site-wide	"You have reached the question limit for this hour…" / "…today's question limit…"
401	Backend	Missing or wrong X-API-Key	Generic "could not answer" (logged in WordPress)
422	Backend	Invalid request body	Generic error
429	Backend	Daily model-request budget reached	"The daily usage cap…"
502	Backend / WordPress	Gemini failure or backend unreachable	"The chat service could not answer right now…"

10. Security

Security is layered, so no single control has to be perfect (Figure 13). An attacker would have to get past Cloudflare, the firewall, Caddy's limits, WordPress's nonce and limits, and the backend's key before any model is called. Even then, the backend's daily budget caps the cost.

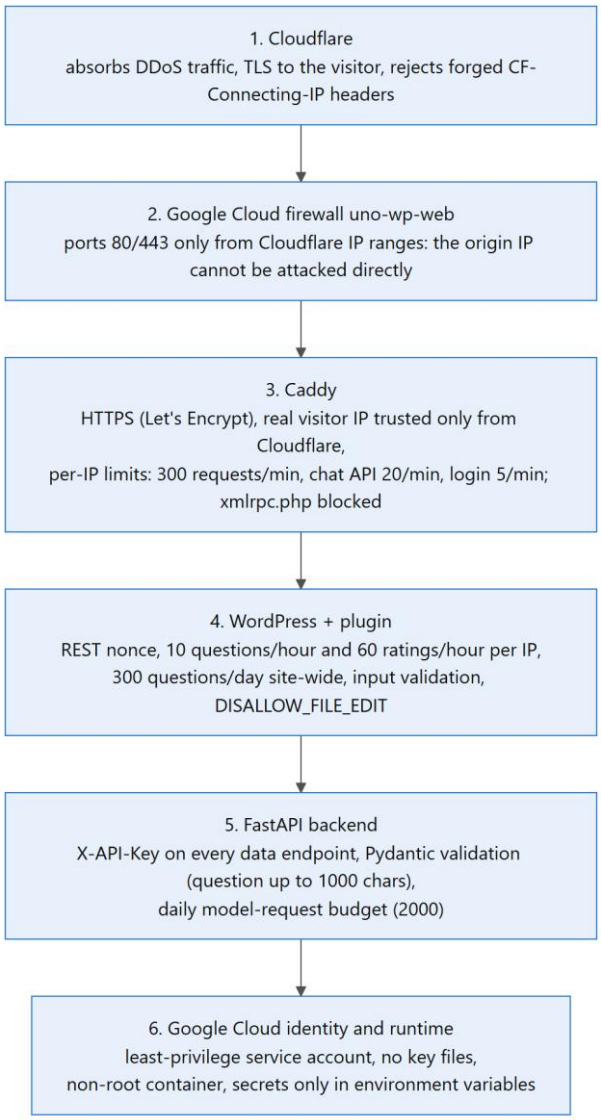


Figure 13. Security layers from the edge to the model

10.1 How API keys and secrets are protected

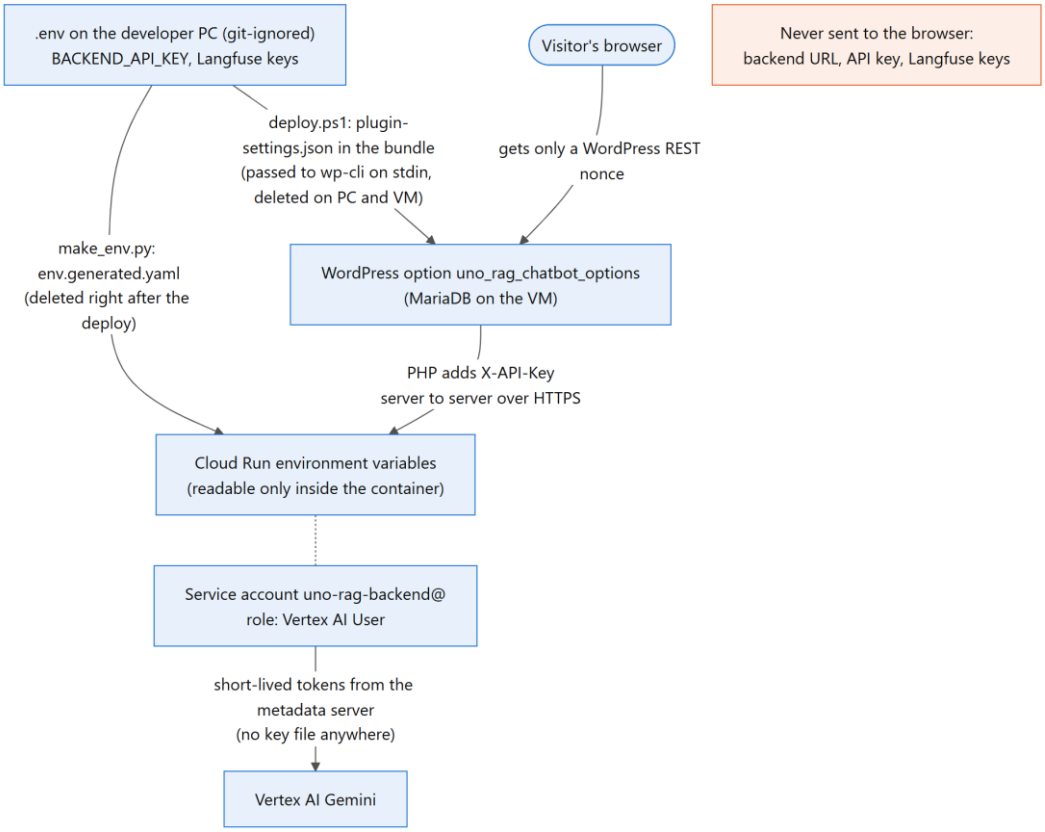


Figure 14. Where secrets live and how they move

Secret	Stored in	Readable by	How it gets there
BACKEND_API_KEY	`.env` (git-ignored); Cloud Run env vars; WordPress option in MariaDB	The developer; the Cloud Run container; WordPress PHP	make_env.py file (deleted after deploy); deploy bundle (deleted on PC and VM, key passed to wp-cli on stdin, never on a command line)
Langfuse public/secret key	`.env`; Cloud Run env vars	The developer; the container	Same env file as above
Google credentials	None stored: Cloud Run uses the service account's short-lived tokens from the metadata server; locally `gcloud auth application-default login`	Google infrastructure	Attached service account uno-rag-backend@ (role Vertex AI User only)
Database passwords	/opt/uno-wp/.env on the VM (mode 600, root only)	Root on the VM; containers	Generated on the VM with openssl, never leave it
WordPress admin password	Hashed in WordPress; plain copy in /opt/uno-wp/admin-credentials.txt (root only) and wordpress/admin-credentials-gcp.txt (git-ignored)	Root on the VM; the developer	Generated on the VM at install

Additional safeguards:

- **Browser:** the browser only ever receives a WordPress nonce. The page HTML contains neither the key nor the backend address, which the security checks verify.
- **/health:** returns model and status information but never configuration secrets, which is also verified.
- **Evaluation config:** the evaluation results store the configuration without any setting whose name contains KEY.

- **Uploads:** `.gcloudignore` and `.dockerignore` keep `.env` and `.venv` out of the Cloud Run upload and the image. The upload list was checked: 24 files.

10.2 Network and DDoS protection

- **Cloudflare proxy:** the DNS record is proxied (orange cloud), so visitors connect to Cloudflare's network. Volumetric attacks are absorbed there.
- **Origin lock-down:** the firewall rule `uno-wp-web` admits ports 80/443 only from Cloudflare's published IP ranges, fetched at deploy time. The VM's own IP doesn't answer web requests from anywhere else, so the protection can't be bypassed.
- **Encryption end to end:** Cloudflare SSL/TLS mode is Full (strict), so Cloudflare verifies the origin certificate. Caddy obtains and renews a Let's Encrypt certificate (HTTP-01 validation passes through Cloudflare). http requests are redirected to https.
- **Backend:** Cloud Run is a Google-managed HTTPS endpoint. It's public, but every data endpoint requires the API key, and it runs at most one instance, which caps cost.

10.3 Rate limiting

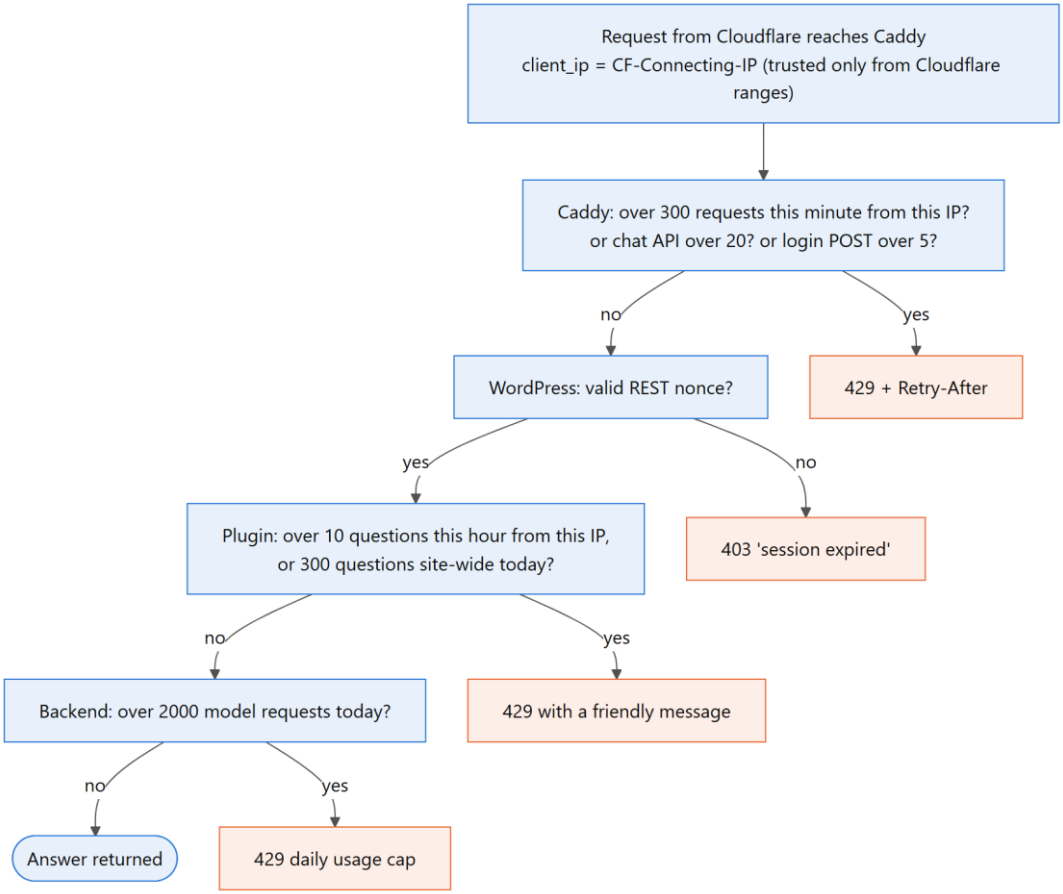


Figure 15. Rate-limiting decisions for one request

Layer	Limit	Key	Response
Caddy zone `site`	300 requests / minute	Visitor IP	429 + Retry-After
Caddy zone `chat`	20 requests / minute to /wp-json/uno-chat/*	Visitor IP	429 + Retry-After
Caddy zone `login`	5 POST /wp-login.php / minute	Visitor IP	429 + Retry-After
Plugin	10 questions / hour (setting)	Visitor IP	429 friendly message

Layer	Limit	Key	Response
Plugin	60 ratings / hour	Visitor IP	429
Plugin	300 questions / day, whole site (setting)	UTC date	429 "today's limit"
Backend	2000 model requests / day	Instance	429 daily cap

> The backend's daily counter lives in the Cloud Run instance's temporary storage. It resets when the instance restarts, about 15 minutes after the last request. The site-wide daily cap in WordPress is kept in MariaDB, so it is the limit that really holds; 300 questions a day is about 900 model requests at most.

10.4 Real visitor IPs and spoofing

Per-IP limits only work if the IP can't be faked. Behind proxies there are three possible sources:

- **The TCP connection:** always Cloudflare.
- `CF-Connecting-IP`: set by Cloudflare.
- `X-Forwarded-For` / `X-Real-IP`: these can be sent by anyone.

The defences:

- Caddy's `trusted_proxies` lists only Cloudflare's ranges, and `client_ip_headers` is `CF-Connecting-IP`. A request that doesn't come from Cloudflare can't choose its IP.
- Cloudflare itself rejects requests that try to send their own `CF-Connecting-IP` (error 1000).
- Caddy sets `X-Real-IP` to the resolved visitor IP and overwrites any client value. WordPress copies it to `REMOTE_ADDR`, so the plugin's limits use the real IP.
- This was tested: 25 chat-API calls with random spoofed `X-Forwarded-For`, `X-Real-IP` and `True-Client-IP` values still hit the 429 at the 21st request in the minute.

10.5 Application hardening

- **WordPress REST nonce** on every chat route (CSRF protection), plus strict argument validation (length, character set, enum).
- **Output encoding:** the widget renders model answers with `textContent`. Admin settings are escaped (`esc_attr`, `esc_html`) and sanitized on save.
- **WordPress hardening:** `DISALLOW_FILE_EDIT` (no theme/plugin editor), `xmlrpc.php` blocked by Caddy, the plugin folder mounted read-only, the WordPress container not published to the internet (only Caddy is).
- **Backend validation:** Pydantic schemas reject oversized or malformed input before any model call (question ≤ 1000 chars, trace id format, feedback value).
- **Containers:** the backend runs as a non-root user on python:3.13-slim. Container logs are capped (10 MB × 3) so floods can't fill the disk. MariaDB is reachable only on the internal Docker network.
- **Grounding:** the answer prompt allows only the retrieved passages, and off-topic requests get a fixed refusal. This limits prompt-injection impact to the rulebook domain.

10.6 Identity and least privilege

- **Cloud Run service account:** `uno-rag-backend@uno-rag-chatbot.iam.gserviceaccount.com` has only **Vertex AI User**. No service-account key exists.
- **VM:** uses the default compute account with the minimal scopes `logging-write` and `monitoring-write`, so the VM can't call other Google APIs.

- **Deploy-time permission grants** (IAM) are a separate, skippable step (`-SkipIamGrants`), so routine updates never change permissions.

10.7 Privacy

- **Backend and Langfuse:** visitors are identified only by a 16-character salted hash of their IP. Raw IPs never leave the VM.
- **Caddy logs:** the access log contains IPs for abuse investigation and is size-capped.
- **Content:** no account, name or email is collected from visitors. Chat content is stored in Langfuse traces for quality monitoring.

10.8 Threats and mitigations

Threat	Mitigation
Stolen backend key from the browser	Key never sent to the browser; only server-to-server
Direct calls to the backend	X-API-Key required; 401 otherwise; daily budget caps cost
DDoS / traffic floods on the site	Cloudflare proxy; firewall admits only Cloudflare; Caddy 300 requests/min per IP; log caps
Bypassing Cloudflare via the origin IP	Firewall rule limited to Cloudflare ranges (origin unreachable, verified)
Credit exhaustion by scripted questions	Caddy 20/min, plugin 10/hour per IP, 300/day site-wide, backend 2000/day, max 1 Cloud Run instance
IP spoofing to escape limits	Trusted-proxy configuration, header overwrite, Cloudflare error 1000 (verified)
Password guessing on wp-admin	5 login POSTs/minute per IP; strong random admin password; xmlrpc blocked
Cross-site request forgery on chat routes	WordPress REST nonce
HTML/script injection via model output	textContent rendering; Markdown stripped
Secrets leaking through deploy artefacts	Generated env files and bundles deleted; ignore files checked; .env git-ignored
Cloud credential theft	No key files; least-privilege service account; minimal VM scopes

10.9 Security test results

`python -m eval.security_checks` ran against the live system on 2026-11-01 02:54: **26 passed, 0 failed, 4 informational**.

Result	Area	Check	Observed
PASS	backend	root is a JSON service description (no web page)	200 application/json
PASS	backend	/health is public and contains no secrets	200, leaked: none
PASS	backend	POST /chat with no key is rejected	401
PASS	backend	POST /chat with wrong key is rejected	401
PASS	backend	POST /ingest with no key is rejected	401
PASS	backend	POST /ingest with wrong key is rejected	401
PASS	backend	POST /feedback with no key is rejected	401
PASS	backend	POST /feedback with wrong key is rejected	401
PASS	backend	DELETE /chat/{session_id} with no key is rejected	401
PASS	backend	DELETE /chat/{session_id} with wrong key is rejected	401
PASS	backend	question over 1000 characters is rejected before any model call	422
PASS	backend	malformed feedback trace id is rejected	422
INFO	backend	OpenAPI docs page	200 (public schema only; every data endpoint still needs the key)
PASS	site	valid HTTPS certificate, served through Cloudflare	200, server=cloudflare, cf-ray=yes

Result	Area	Check	Observed
PASS	site	http redirects to https	308 -> https://uno-chatbot.buildwithmayank.tech/
PASS	network	origin 34.57.222.238:443 not reachable directly (Cloudflare-only firewall)	no answer within 5s
PASS	network	origin 34.57.222.238:80 not reachable directly (Cloudflare-only firewall)	no answer within 5s
PASS	site	xmlrpc.php blocked	403
INFO	site	security header: HSTS	not set
INFO	site	security header: nosniff	not set
INFO	site	security header: frame options	not set
PASS	plugin	chat page does not expose the backend API key	absent
PASS	plugin	chat page does not expose the backend address	absent
PASS	plugin	REST /ask without a nonce is rejected	403
PASS	plugin	REST /ask with a forged nonce is rejected	403
PASS	plugin	REST /ask with the page's nonce is answered (WordPress -> backend proxy works)	200
PASS	rate-limit	forged CF-Connecting-IP header is rejected at Cloudflare's edge	403 Cloudflare error 1000
PASS	rate-limit	chat API: per-IP burst limit (20/min) holds even with spoofed IP headers	first 429 at call 18 of this burst (+3 earlier), Retry-After 53s, statuses [403, 429]
PASS	rate-limit	login: 5 attempts/min per IP, then 429 with Retry-After	attempt statuses [200, 200, 200, 200, 200, 429], Retry-After 59s
PASS	rate-limit	login page itself still loads while POSTs are limited	200

10.10 Known limitations and recommendations

- **Security headers:** HSTS, `X-Content-Type-Options` and `X-Frame-Options` aren't set yet. Add them in Caddy (or as a Cloudflare transform rule).
- **Secrets:** they are plain environment variables (by choice). For stricter control, move `BACKEND_API_KEY` and the Langfuse keys to Secret Manager.
- **SSH:** port 22 on the VM is open to the internet (default rule; key-based login only). Restrict it to Google's IAP range and use `gcloud compute ssh --tunnel-through-iap`.
- **Backend budget:** it resets per instance (section 10.3); a persistent counter (Firestore) would make it exact.
- **IPv6 visitors:** an IPv6 visitor can rotate addresses within their /64 block. Keying the Caddy limits on the /64 prefix would close this.
- **API docs:** `/docs` is public (schema only). Disable it in production if the API surface should not be advertised.
- **Backups:** schedule weekly snapshots of the VM disk.

11. Deployment

Production runs in the Google Cloud project `uno-rag-chatbot` behind the Cloudflare zone `buildwithmayank.tech` (Figure 16). Two idempotent PowerShell scripts deploy everything from the developer's Windows PC. They are safe to re-run, and each run is also an update.

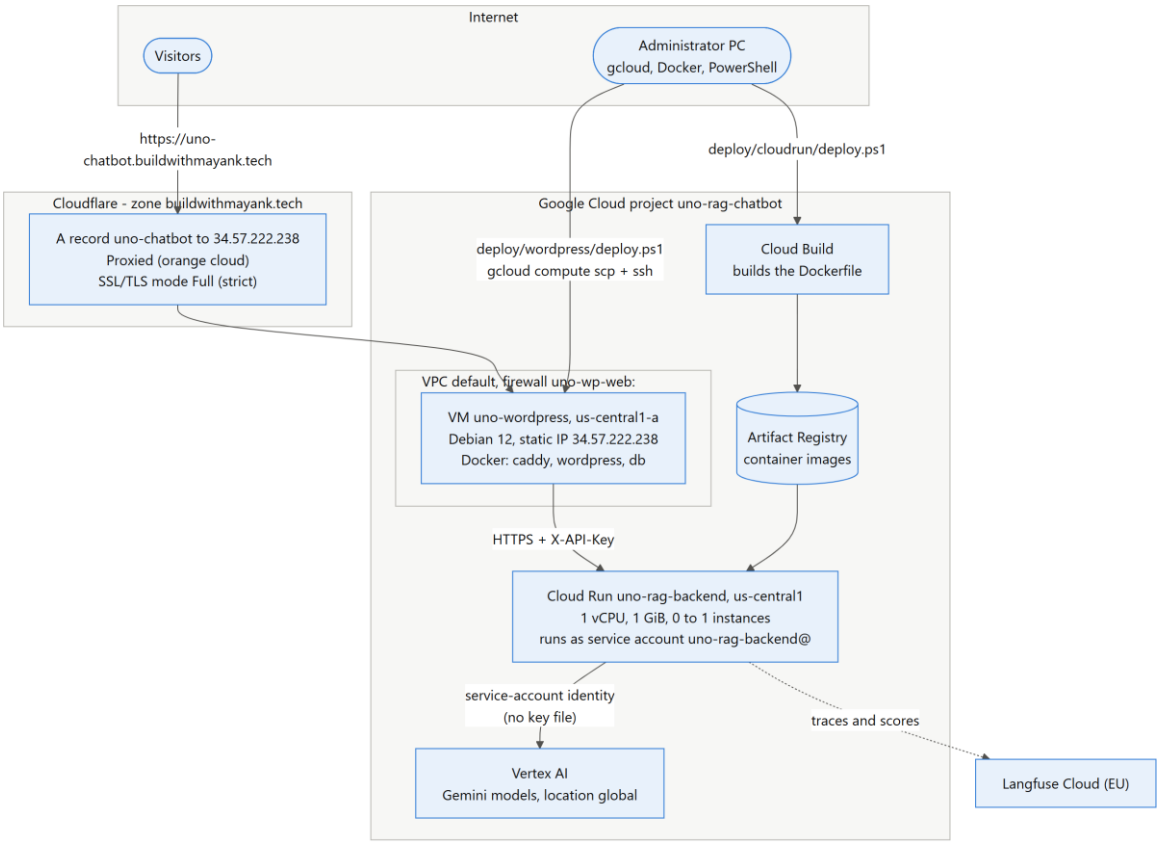


Figure 16. Deployment topology

11.1 Backend on Cloud Run

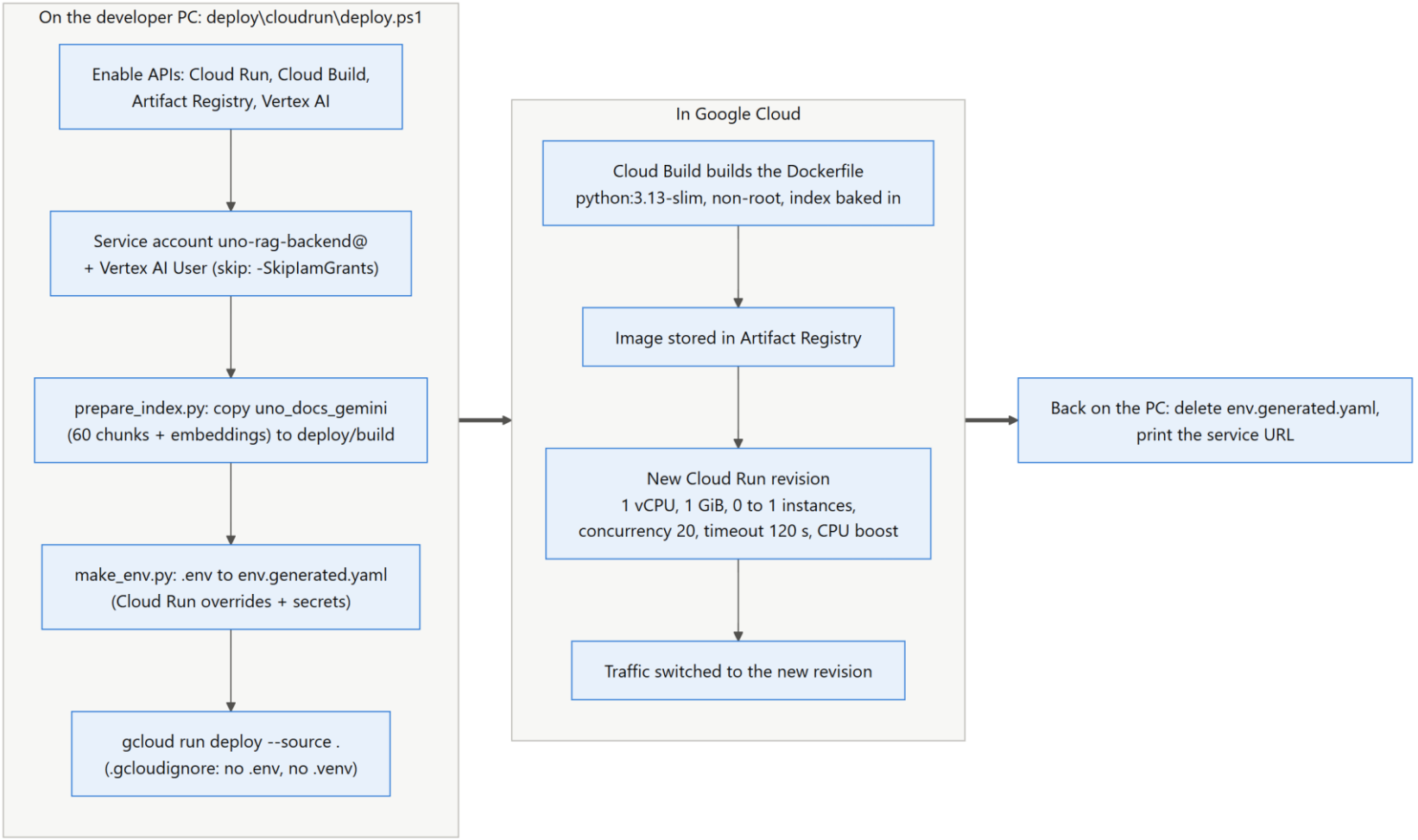


Figure 17. Backend deployment workflow (deploy/cloudrun/deploy.ps1)

Setting	Value	Why
Region	us-central1	Low latency to the VM; Vertex AI location is global
CPU / memory	1 vCPU / 1 GiB	Enough for FastAPI + Chroma with 60 chunks
Instances	min 0, max 1	Free when idle; one instance caps cost and keeps memory/caches consistent
Concurrency / timeout	20 requests / 120 s	Model calls are I/O-bound; 120 s covers retries
CPU boost	on	Faster cold starts
Identity	service account uno-rag-backend@ (Vertex AI User)	No key file needed
Ingress / auth	public, --allow-unauthenticated	Protected by X-API-Key instead of Google IAM (WordPress isn't a Google identity)
Image	python:3.13-slim, non-root user, index + PDFs baked in	Fast, deterministic start-up; no re-embedding
Env overrides	CACHE_DIR=/tmp/uno-cache, CHROMA_DIR=/app/chroma_db, LANGFUSE_ENVIRONMENT=production, LANGFUSE_FLUSH_EACH_REQUEST=true	Writable temp storage; traces sent before the response ends (CPU is throttled afterwards)

11.2 WordPress on Compute Engine

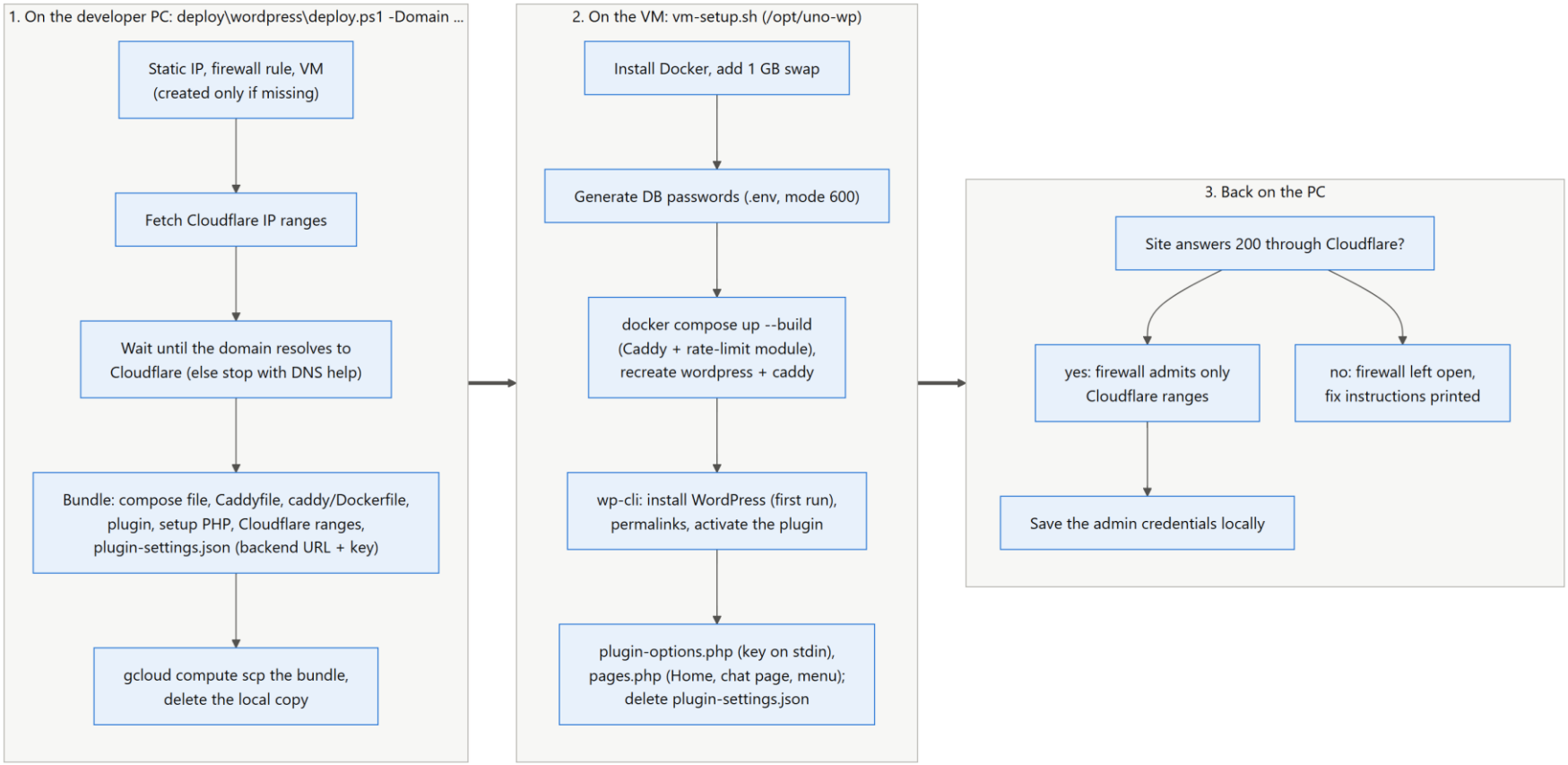


Figure 18. WordPress deployment workflow (deploy/wordpress/deploy.ps1)

Item	Value
VM	uno-wordpress, e2-small (2 vCPU burst, 2 GB) + 1 GB swap, Debian 12, 20 GB pd-balanced, shielded secure boot, us-central1-a
Network	Static IP 34.57.222.238; firewall uno-wp-web: tcp 80/443 (+udp 443) from Cloudflare ranges only; tag uno-wp
Containers (/opt/uno-wp)	caddy (built with caddy-ratelimit, ports 80/443), wordpress:php8.3-apache (internal only), mariadb:11 (innodb buffer 128 MB, performance schema off), wpcli (tools profile)
Volumes	db_data, wp_data, caddy_data (certificates), caddy_config
Configuration	WORDPRESS_CONFIG_EXTRA: REMOTE_ADDR from X-Real-IP, DISALLOW_FILE_EDIT, WP_MEMORY_LIMIT 128M
Updates	Re-run deploy.ps1 -Domain …: new plugin files, settings kept, wordpress + caddy recreated (data in volumes)

11.3 Cloudflare configuration

1. **DNS → Records:** an A record `uno-chatbot` → 34.57.222.238 with Proxy status **Proxied** (orange cloud).
2. **SSL/TLS → Overview:** encryption mode **Full (strict)**. Flexible causes a redirect loop, because Caddy redirects http to https.
3. **Optional:** Security → Bots → Bot Fight Mode, and a WAF rate-limiting rule (the free plan includes one).

11.4 Updates and rollback

Change	Command / action
Backend code or settings	`deploy\cloudrun\deploy.ps1 -SkipIamGrants` (new revision)
New or changed PDFs	`python -m app.ingest --reset`, run the evaluation, then deploy the backend
Plugin or site setup	`deploy\wordpress\deploy.ps1 -Domain uno-chatbot.buildwithmayank.tech`
Backend rollback	`gcloud run services update-traffic uno-rag-backend --region us-central1 --to-revisions <revision>=100`
Site rollback / recovery	Restore the VM disk snapshot, or redeploy the previous plugin version

11.5 Cost

Item	Monthly cost (approx.)
VM e2-small	$13
Static IP	$3 (also billed while the VM is stopped)
20 GB balanced disk	$2
Cloud Run	$0 (free tier at this traffic)
Cloud Build + Artifact Registry	a few cents
Gemini (pay per use)	a fraction of a cent per question; a full evaluation run of ~100 requests costs a few cents
Cloudflare free plan, Langfuse Hobby	$0

12. Monitoring and observability

When the Langfuse keys are set, the backend records one trace per chat turn. Without them, the chat works the same and nothing is sent.

- **Trace** `chat`, one per turn:
  - **Grouping:** the session (the conversation) and the user (the anonymous visitor hash).
  - **Input and output:** the question as input; the answer and sources as output.
  - **Metadata:** route, found, rewrites, standalone question, cached, the models, and the requests used today.
- **Child observations:** every graph node, every Gemini call (with the actual model, tokens and latency), the query embedding and the retrieved chunks, via the LangChain callback handler.
- **Tags:** `route:rulebook`, `route:chitchat`, `route:off_topic`, `cached`, `refused`, `daily-limit` and `error`, for filtering and dashboards.
- **Environments:** `local` (development), `production` (Cloud Run) and `eval` (evaluation runs).
- **Scores:** `user-feedback` (BOOLEAN) from the widget's thumbs. The evaluation adds `answer_correct`, `citation_correct`, `route_correct` and `requests_used` in experiments on the dataset `uno-rulebook-qa`, plus per-method retrieval scores.
- **Serverless delivery:** on Cloud Run each request flushes its trace before returning, because the CPU is throttled after the response.

Operational logs:

- **Cloud Run:** `gcloud run services logs read uno-rag-backend --region us-central1`.
- **VM:** `sudo docker compose logs caddy|wordpress|db` (Caddy logs JSON access lines with the visitor IP and status).
- **Usage:** the plugin settings page shows backend health, requests used today and site questions today.

13. Evaluation

The evaluation uses a hand-labelled test set written against the extracted rulebook text (`eval/dataset.py`):

- **31 rulebook questions:** each lists the facts the answer must contain (for example "25" for the Mercy rule) and the correct source PDF.
- **6 off-topic questions:** these must be refused without sources.
- **16 small-talk messages:** these must be answered without retrieval.
- **4 two-turn conversations:** the second question only makes sense with the first.

Three runs measure the system:

- **Local:** retrieval metrics and end-to-end answers on the local graph, also logged as a Langfuse experiment.
- **Production:** the same questions over HTTPS against the deployed Cloud Run API.
- **Security:** automated checks of the live security measures (section 10.9).

```
python -m eval.run_eval                     # -> eval/results_gemini.json
python -m eval.run_eval --backend-url URL   # -> eval/results_production.json
python -m eval.security_checks              # -> eval/results_security.json
```

13.1 Retrieval

See section 5 for the method comparison. With the production hybrid search and rulebook filter, the correct rulebook is ranked first for 96.8% of questions, and the passage containing the answer is in the top 6 for 96.8%.

13.2 End-to-end accuracy

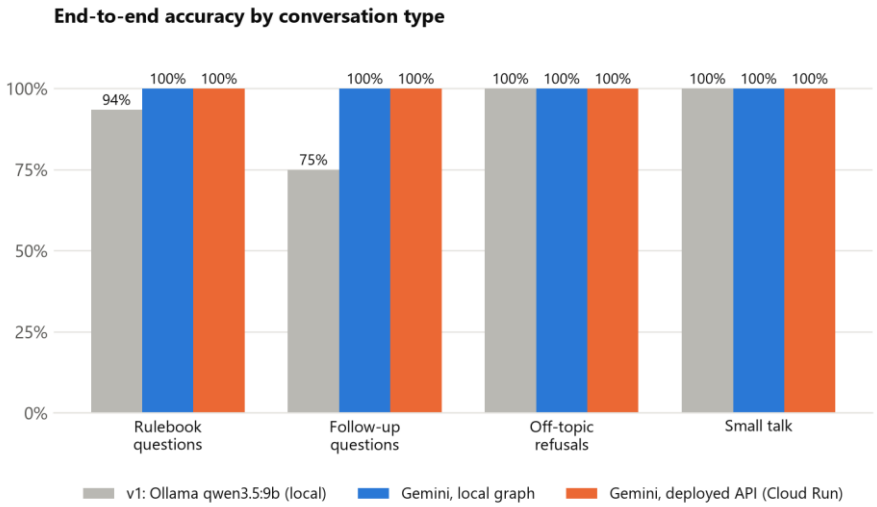


Figure 19. End-to-end accuracy: first version vs. current system (local and deployed)

Group	Local graph	Deployed API	How it is scored
Rulebook questions	31/31 (100%)	31/31 (100%)	Every required fact appears in the answer
Correct citation	100%	100%	A source chip names the correct PDF
Off-topic	6/6	6/6	Refusal text and no sources
Small talk	16/16	16/16	Answered without retrieval or refusal
Follow-ups	4/4	4/4	Second answer contains the required facts
Routing (local only)	100%	–	Router chose the expected route

13.3 Latency

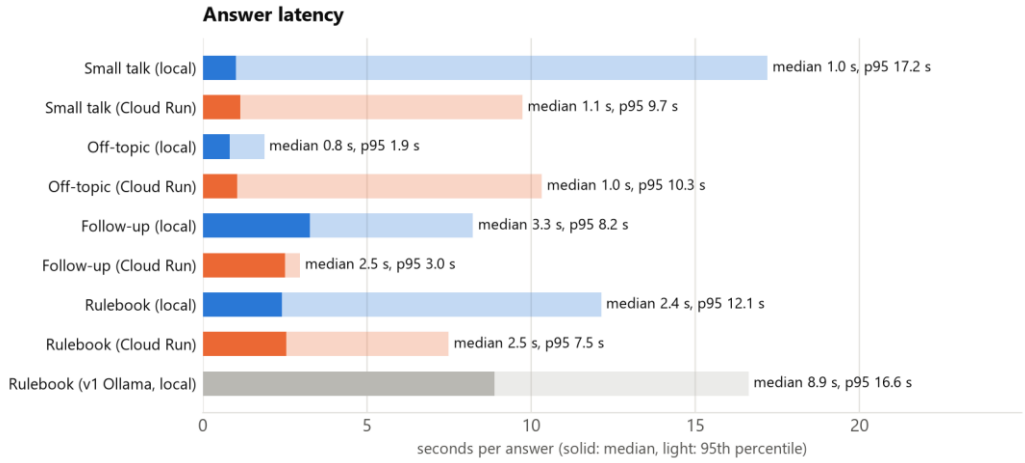


Figure 20. Answer latency by message type

Medians are stable: 2.4 s locally and 2.5 s on Cloud Run for rulebook questions, and about 1 s for small talk. Only 1 of the 57 production answers came from the answer cache, so these are real model latencies.

The 95th percentiles are dominated by a few slow calls on the model side. The slowest answers took 21.4 s locally and 32.4 s on Cloud Run. During the local run, Vertex AI answered one request with HTTP 429 (shared capacity), and the backend retried it after 5 seconds; other slow calls were queued at Vertex AI. The backend's retry logic absorbs these without failing the answer.

13.4 Project evolution

The first version (October 2026) ran entirely on the developer's PC with Ollama (qwen3.5:9b for answers, nomic-embed-text for embeddings). The current version uses Gemini on Vertex AI and was redesigned in two ways:

- **One router call** replaced the separate classify, contextualize and grade steps.
- **Retrieval** gained the contextual rulebook header and query caching.

Measure	v1 Ollama (local)	v2 Gemini (local)	v2 Gemini (deployed)
Rulebook answers correct	93.5%	100%	100%
Correct rulebook cited	96.8%	100%	100%
False refusals	1	0	0
Follow-ups correct	75%	100%	100%
Off-topic refused	100%	100%	100%
Median time, rulebook question	8.9 s	2.4 s	2.5 s
Answer chunk in top 6 (production retrieval)	93.5%	96.8%	same index
MRR of the answer chunk	0.528	0.836	same index

14. Setting up the project from scratch

Figure 21 (after the prerequisites) shows the order of the setup steps; the commands below are complete.

14.1 Prerequisites

Need	Notes
Windows 10/11 PC with PowerShell	The deploy scripts are PowerShell; the VM script is bash and runs on the VM
Python 3.13	Backend, ingest, evaluation
Docker Desktop	Local WordPress (and optional local container test)
Google Cloud SDK (gcloud)	Logged in with `gcloud auth login` and `gcloud auth application-default login`
Google Cloud project with billing	Trial credits work; the deploy script enables the needed APIs
A domain on Cloudflare (free plan)	For the public site; a subdomain is enough
Langfuse Cloud account (optional)	Free Hobby plan; project API keys
Node.js (optional)	Only to rebuild this report

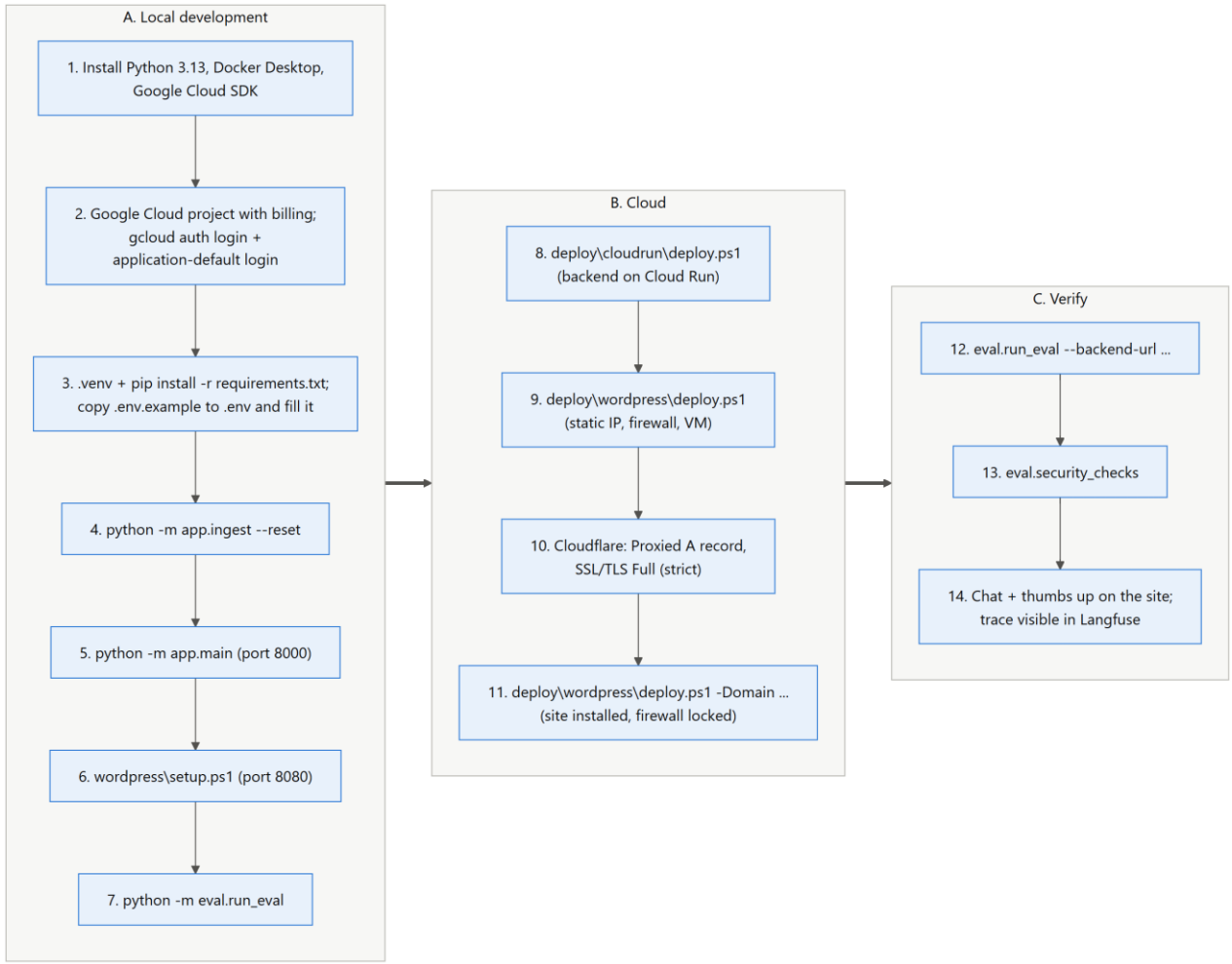


Figure 21. Setup from scratch

14.2 Local backend

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
#   edit .env: GOOGLE_CLOUD_PROJECT=<project id>
#              BACKEND_API_KEY=<long random string>
#              LANGFUSE_* keys (optional)
gcloud auth application-default login
gcloud services enable aiplatform.googleapis.com --project <project id>
python -m app.model_status          # checks the Gemini models (free)
python -m app.ingest --reset        # builds chroma_db/ from Data/*.pdf
python -m app.main                  # http://localhost:8000/health
```

14.3 Local WordPress

```
# Docker Desktop running
powershell -ExecutionPolicy Bypass -File wordpress\setup.ps1
#   -> http://localhost:8080, chat at /uno-rules-chat/
#      admin login in wordpress\admin-credentials.txt
python -m eval.run_eval              # optional: accuracy check (~100 Gemini requests)
```

14.4 Backend on Cloud Run

```
gcloud config set project <project id>
powershell -ExecutionPolicy Bypass -File deploy\cloudrun\deploy.ps1
#   prints the service URL; check  https://<service>/health
#   later updates:  deploy\cloudrun\deploy.ps1 -SkipIamGrants
```

The first deploy creates the service account and grants it Vertex AI User. On new projects it also lets the default compute account run source builds. If the first build fails with a permission error, wait two minutes for the grant to apply and run the script again.

14.5 WordPress on Compute Engine with Cloudflare

```
powershell -ExecutionPolicy Bypass -File deploy\wordpress\deploy.ps1
#   creates static IP + firewall + VM and prints the IP
# Cloudflare: DNS A record <name> -> <IP>, Proxied; SSL/TLS mode Full (strict)
powershell -ExecutionPolicy Bypass -File deploy\wordpress\deploy.ps1 -Domain <domain>
#   installs everything, then locks the firewall to Cloudflare
#   admin login saved to wordpress\admin-credentials-gcp.txt
```

14.6 Verify

```
python -m eval.run_eval --backend-url https://<service>.run.app
python -m eval.security_checks --site https://<domain> \
    --backend https://<service>.run.app --origin-ip <VM IP>
```

Then open the site, ask a question and rate the answer. Check that the trace and the `user-feedback` score appear in Langfuse.

15. Operations and troubleshooting

Task	Command
Backend health	`curl https://<service>/health`
Backend logs	`gcloud run services logs read uno-rag-backend --region us-central1`
SSH to the VM	`gcloud compute ssh uno-wordpress --zone us-central1-a`
Site logs / status	`cd /opt/uno-wp && sudo docker compose ps` / `logs -f caddy`
Blocked requests	`sudo docker compose logs caddy | grep '"status":429'`
Stop / start the VM	`gcloud compute instances stop|start uno-wordpress --zone us-central1-a`
Plugin limits and usage	WordPress → Settings → UNO Chatbot

Symptom	Cause	Fix
"Your session expired" (403)	Old nonce in a cached page	Reload; exclude the chat page from full-page caching
"question limit for this hour / today" (429)	Plugin limits reached	Wait, or raise them in the settings
429 with Retry-After on pages	Caddy per-IP limit	Wait one minute
Too many redirects	Cloudflare SSL mode Flexible	Set Full (strict)
Cloudflare 521/522	VM stopped or firewall wrong	Start the VM; re-run the WordPress deploy
Raw shortcode text on the site	Plugin files not visible in the container	Re-run the WordPress deploy (recreates the containers)
"Backend not reachable" in settings	Wrong URL or cold start	Check the URL; retry after a few seconds
Backend 401 in WordPress logs	Keys differ	Re-run the WordPress deploy (copies the key from .env)
Backend 502	Gemini auth/quota	`/health` → gemini.ok; check credentials and quotas

Appendix A. Configuration reference (.env)

Setting	Default	Description
GOOGLE_CLOUD_PROJECT	–	Google Cloud project for Vertex AI
GOOGLE_CLOUD_LOCATION	global	Vertex AI location
GOOGLE_APPLICATION_CREDENTIALS	empty	Optional key file; empty = Application Default Credentials
ROUTER_MODEL / ROUTER_THINKING	gemini-3.5-flash-lite / minimal	Routing, follow-up rewriting, small talk, query rewrite
ANSWER_MODEL / ANSWER_THINKING	gemini-3.5-flash / low	Grounded answers
LLM_TEMPERATURE / LLM_MAX_TOKENS	0.1 / 2000	Sampling (ignored by Flash-Lite) and output limit incl. thinking
EMBED_MODEL / EMBED_DIMENSIONS	gemini-embedding-2 / 768	Embedding model and vector size
EMBED_QUERY_PREFIX / EMBED_DOC_PREFIX	task: search result | query: / title: none | text:	Task prefixes for gemini-embedding-2
EMBED_CONCURRENCY	8	Parallel embedding requests during ingest
DAILY_REQUEST_BUDGET	2000	Model requests per UTC day before 429
ANSWER_CACHE / CACHE_DIR	true / cache	First-turn answer cache and its folder
DATA_DIR / CHROMA_DIR / COLLECTION_NAME	Data / chroma_db / uno_docs_gemini	Inputs and index
CHUNK_SIZE / CHUNK_OVERLAP	1000 / 150	Chunking
TOP_K / MAX_REWRITES / HISTORY_WINDOW	6 / 1 / 6	Chunks per answer, query retries, history messages
RETRIEVAL_MODE	hybrid	hybrid or bm25 (no query embedding)
BACKEND_API_KEY	–	Shared secret for X-API-Key
LANGFUSE_PUBLIC_KEY / SECRET_KEY / BASE_URL	empty / empty / https://cloud.langfuse.com	Monitoring (off when keys are empty)
LANGFUSE_ENVIRONMENT / LANGFUSE_FLUSH_EACH_REQUEST	local / false	production / true on Cloud Run
API_HOST / API_PORT	0.0.0.0 / 8000	Local server (Cloud Run uses $PORT)
AUTO_INGEST_ON_STARTUP / LOG_LEVEL	true / INFO	Ingest when the collection is empty; logging

Appendix B. Per-question results

Every end-to-end test case, run on the local graph and against the deployed API (✓ correct, ✗ wrong; times in seconds). Follow-ups show both turns.

#	Question	Local	s	Deployed	s
1	How many cards are in the UNO Dare deck?	✓	2.0	✓	2.6
2	What are the Dare List categories in UNO Dare?	✓	2.4	✓	2.5
3	What happens when a Dare card is played in UNO Dare?	✓	3.1	✓	3.0
4	How many points is a Wild Dare card worth in UNO Dare?	✓	21.3	✓	2.4
5	What happens if a player fails a dare in UNO Dare?	✓	12.1	✓	2.6
6	How many points do you need to win UNO Dare?	✓	2.4	✓	32.4
7	How many cards are in UNO No Mercy?	✓	1.9	✓	2.3
8	What is the Mercy rule in UNO No Mercy?	✓	2.6	✓	2.8
9	How does stacking work in UNO No Mercy?	✓	3.1	✓	3.8
10	What happens when you play a 7 in UNO No Mercy?	✓	2.1	✓	2.6
11	What happens when a 0 is played in UNO No Mercy?	✓	1.8	✓	2.3
12	What does the Wild Color Roulette card do?	✓	4.2	✓	2.7
13	How many points do you get for knocking out a player in UNO No Mercy?	✓	2.3	✓	2.4
14	How many points are needed to win UNO No Mercy when keeping score?	✓	2.2	✓	2.5
15	What does the Discard All card do in UNO No Mercy?	✓	2.3	✓	2.5
16	How many cards are in UNO Flip?	✓	1.8	✓	2.5
17	What does the Flip card do in UNO Flip?	✓	2.7	✓	4.0
18	What does the Draw Five card do in UNO Flip?	✓	2.2	✓	3.1
19	What does the Skip Everyone card do in UNO Flip?	✓	2.7	✓	0.5
20	How many points is the Wild Draw Color card worth in UNO Flip?	✓	2.6	✓	2.4
21	In UNO Flip, what happens if you challenge a Wild Draw 2 and the player is innocent?	✓	2.2	✓	2.0
22	What colors are on the dark side of UNO Flip?	✓	3.1	✓	3.0
23	How many points is the Skip Everyone card worth in UNO Flip?	✓	2.3	✓	2.4
24	How many cards are dealt to each player in UNO All Wild?	✓	1.9	✓	3.0
25	What does the Wild Skip Two card do in UNO All Wild?	✓	2.5	✓	2.5
26	What does the Wild Forced Swap card do?	✓	2.4	✓	2.8
27	What does the Wild Targeted Draw Two card do?	✓	2.1	✓	2.2
28	In UNO All Wild, do you need to match the color or number?	✓	2.5	✓	2.3
29	How many points do you need to win UNO All Wild?	✓	2.3	✓	2.5
30	What is the penalty for forgetting to call UNO?	✓	2.9	✓	7.5
31	Which card makes the next player draw until they get a chosen color?	✓	5.0	✓	2.7
32	What is the capital of France?	✓	1.9	✓	1.1
33	Who won the 2018 FIFA World Cup?	✓	0.8	✓	1.1
34	Write a Python function to reverse a string.	✓	0.9	✓	1.0
35	How do I bake a chocolate cake?	✓	0.8	✓	1.0
36	How does castling work in chess?	✓	0.7	✓	10.3
37	What is the boiling point of water?	✓	0.8	✓	0.9
38	hi	✓	1.0	✓	1.1
39	hello there	✓	1.0	✓	1.2
40	hey!	✓	1.1	✓	1.1

#	Question	Local	s	Deployed	s
41	good morning	✓	1.0	✓	1.1
42	how are you?	✓	17.2	✓	9.7
43	how's it going?	✓	1.1	✓	1.2
44	thanks!	✓	1.2	✓	1.0
45	thank you so much	✓	0.8	✓	1.1
46	bye	✓	1.0	✓	0.9
47	see you later	✓	1.2	✓	1.1
48	who are you?	✓	0.9	✓	1.5
49	what can you do?	✓	1.1	✓	1.1
50	what is your name?	✓	0.9	✓	1.2
51	nice to meet you	✓	1.2	✓	5.2
52	ok cool	✓	0.9	✓	1.2
53	you're awesome	✓	0.9	✓	1.1
54	What does the Skip Everyone card do in UNO Flip? -> How many points is it worth?	✓	2.4	✓	2.5
55	How many cards are dealt in UNO No Mercy? -> Is it the same in UNO Dare?	✓	3.7	✓	2.5
56	What is the Mercy rule in UNO No Mercy? -> How many points do you get for knocking someone out?	✓	2.8	✓	3.0
57	What does the Wild Draw Color card do in UNO Flip? -> What happens if I challenge it and the player was innocent?	✓	8.2	✓	2.5

Appendix C. Diagram sources

All diagrams are Mermaid files in `docs/diagrams/`, rendered to PNG by `docs/tools/render_diagrams.py`. The charts come from `docs/tools/charts.py` and the evaluation JSON files. To rebuild this report, run `node docs/tools/build_report.js`.

File	Figure	Shows
`01_architecture.mmd`	1	Components and how they connect
`02_deployment_topology.mmd`	16	Cloudflare, Google Cloud resources, deploy paths
`03_ingestion_pipeline.mmd`	3	PDF to Chroma indexing steps
`04_retrieval_pipeline.mmd`	6	Hybrid search and reciprocal rank fusion
`05_langgraph_flow.mmd`	8	LangGraph nodes, routes and memory
`06a_chat_sequence_frontend.mmd`	9	Chat request from the browser to WordPress
`06b_chat_sequence_backend.mmd`	10	Chat request from WordPress to Gemini
`07_feedback_sequence.mmd`	11	Thumbs up/down to a Langfuse score
`08_api_key_flow.mmd`	14	Where secrets live and how they move
`09_security_layers.mmd`	13	The six security layers
`10_rate_limiting_flow.mmd`	15	Rate-limit decisions per request
`11_frontend_components.mmd`	12	WordPress pages, plugin files and assets
`12_deploy_backend.mmd`	17	Cloud Run deployment workflow
`13_deploy_wordpress.mmd`	18	VM deployment workflow
`14_setup_from_scratch.mmd`	21	Setup order from a new machine
`15_data_flow.mmd`	2	Indexing and answering data flow
`16_chroma_schema.mmd`	5	ChromaDB storage schema (entity-relationship)
`17_data_stores.mmd`	4	All data stores and data movement